

Week15:

Exploring Azure Databricks Interface:

To use databricks we need to create a databricks workspace.

An Azure Resource Group is a logical container used to **group, deploy, manage, and monitor related Azure resources**. Think of it as a folder for your cloud infrastructure.

RBAC Inheritance: Assign permissions at the resource group level instead of per individual asset.

Simultaneous Deletion: Delete a resource group to instantly wipe out all contained assets.

Step1: Resource group in azure

<project>-<env>-<region>-rg

Ex: Databricks-training-centralindia-rg

Step2: search for databricks

Workspace name - <project>-<env>-<region>-dbws

Ex: training-databricks-centralindia-dbws

Price Tier: 3 types

Managed Resource Group Name : when we create a cluster in databricks in backend it will run VM's on azure. So in which resource group these backend resources should resides.

Ex: training-centralindia-managed-rg

When you use databricks then how is your cost determined ?

3 node cluster

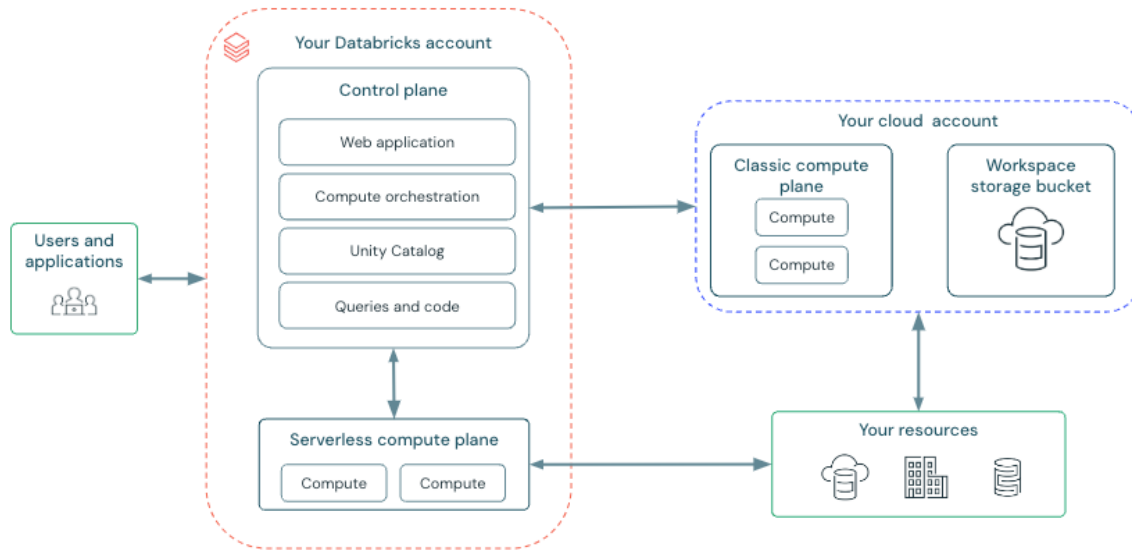
The cost of these 3 vms + service cost

Infrastructure cost (azure) (service provider)

Software cost (Databrick) in the way of DBU.

Hands-on with Databricks Notebooks:

Display(dbutils.fs.ls("/databricks-datasets/airlines"))



Control Plane (Databricks cloud account)

- Metadata and ACL
- Configurations (cluster config, notebook config, job info)

We can access our notebook through URL as well. Every time we don't need to login through azure databricks workspace.

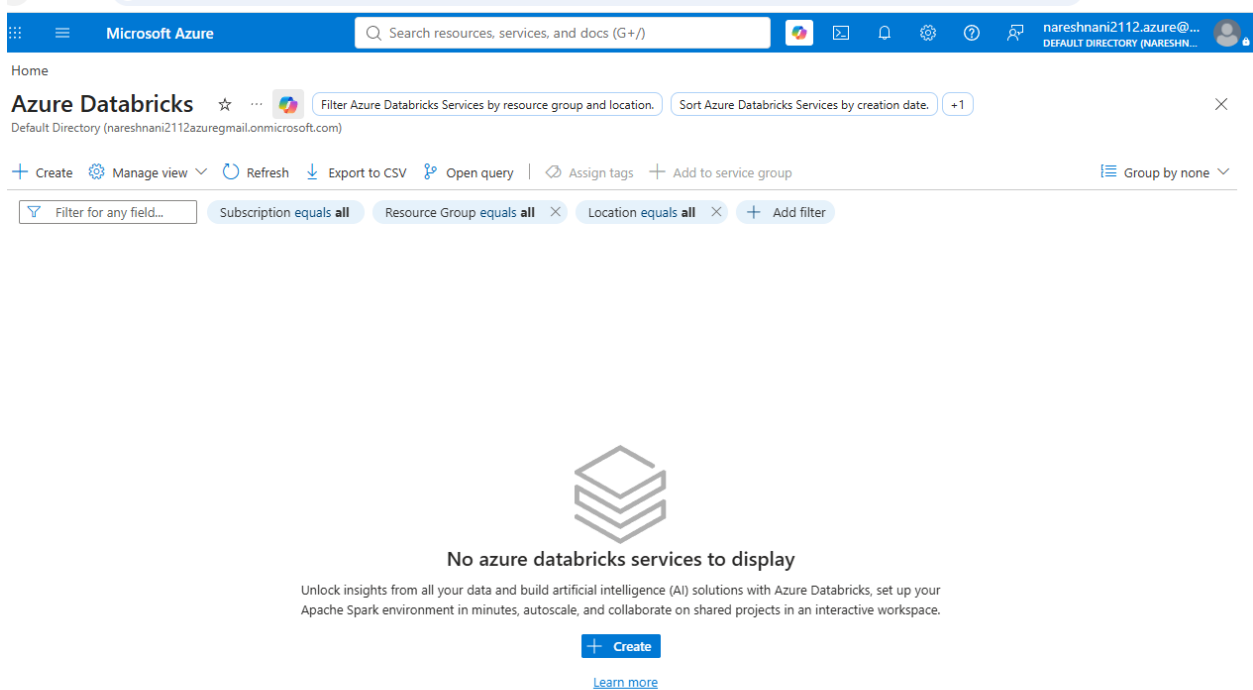
Data plane / compute plane(customer cloud account)

Client data

Classic compute cluster

Once we create a classic compute aa compute details manam manam create chesina managed resource group folder lo chudavachu. (training-centralIndia-managed-rg)...

We can create multiple workspace



When ever we create a workspace we will get one metastore and default catalog (with default and information_schema).

Managing Table Data and Metadata in Databricks Using Unity Catalog

Select current_metastore();

Select current_catalog();

Select current_database();

Show catalogs; → it gives all the catalogs.

When you create a table in Databricks without specifying a custom LOCATION, the data is stored in the **DBFS (Databricks File System) root**, specifically within the **managed warehouse directory**.

1. With Unity Catalog (Modern Default)

If your workspace uses Unity Catalog, the data is stored in the managed storage location defined for the schema, catalog, or metastore. By default, it follows this hierarchy:

- **Path:** abfss://<container>@<storage-account>.dfs.core.windows.net/metadata/unity-catalog/managed/...
- The data is automatically isolated by catalog and schema name.

2. Without Unity Catalog (Legacy Hive Metastore)

If you are using the legacy workspace local Hive metastore, the data is stored in the default DBFS root warehouse directory:

- **Path:** dbfs:/user/hive/warehouse/<schema_name>.db/<table_name>

```
%sql
create table if not exists hive_metastore.default.students
(
  Id int,
  Name string,
  Score Float
);
```

```
%sql
describe EXTENDED hive_metastore.default.orders_demo_112;
```

When we run above query we can see the catalog value as : **hive_metastore**

How to Use Azure Storage with Databricks Unity Catalog

Microsoft Azure

Search resources, services, and docs (G+/)

Home > Storage center | Blob Storage

Create a storage account

Basics Advanced Networking Data protection Security Encryption Tags Review + create

Azure Storage is a Microsoft-managed service providing cloud storage that is highly available, secure, durable, scalable, and redundant. Azure Storage includes Azure Blobs (objects), Azure Data Lake Storage Gen2, Azure Files, Azure Queues, and Azure Tables. The cost of your storage account depends on the usage and the options you choose below. [Learn more about Azure storage accounts](#)

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * Azure subscription 1

Resource group * databricks_learning_demo
[Create new](#)

Instance details

Storage account name * ① databricksdemo10121

Region * ① (Asia Pacific) Central India
[Deploy to an Azure Extended Zone](#)

Primary service * Azure Blob Storage or Azure Data Lake Storage

Performance * ①

☒ Standard: Recommended for most scenarios (general-purpose v2 account)

☐ Premium: Recommended for scenarios that require low latency.

Redundancy * ① Locally redundant storage (LRS)

Storage account: databricksdemo10121

Blob container: unity catalog

Folder (directroy): catalog

I want databricks to access above storage.. for that we need access connector..

An **Access Connector for Azure Databricks** is a first-party Azure resource that acts as a secure bridge or wrapper around an Azure Managed Identity. Its primary purpose is to allow Azure Databricks to connect to secure Azure resources (like cloud storage) without needing hardcoded access keys or service principal secrets in your code.

It is most commonly used to set up secure governance inside Unity Catalog

Microsoft Azure

Search resources, services, and docs (G+/)

Home > Access Connector for Azure Databricks

myaccessconnector
Access Connector for Azure Databricks

Search Refresh Delete

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Settings
- Automation
- Help

Essentials

Resource group ([move](#))
[databricks_learning_demo](#)

Location
Central India

Subscription ([move](#))
[Azure subscription 1](#)

Subscription ID
e4c7c548-c8af-40cc-9c22-a9f095c3c8f5

Tags ([edit](#))
[Add tags](#)

State
Succeeded

Resource ID
/subscriptions/e4c7c548-c8af-40cc-9c22-a9f095c3c8f5/resourceGroups/da...

JSON View

Once we create access connector for databricks. We need to give access to storage account.

In the storage account: iam roles add roles → storage blob contributor then add → select member (access connector for databricks) connector_name:

Microsoft Azure

Search resources, services, and docs (G+/)

Home > Storage center | Blob Storage > databricksdemo10121 | Access Control (IAM)

Add role assignment

Role **Members** Conditions Review + assign

Selected role
Storage Blob Data Contributor

Assign access to
☒ User, group, or service principal
☐ Managed identity

Members
[+ Select members](#)

Name	Object ID	Type
No members selected		

Description
Optional

Select members

myaccess

myaccessconnector
/subscriptions/e4c7c548-c8af-40cc-9c22-a9f095c3c8f5/resourcegroups/data...

Selected members:

myaccessconnector
/subscriptions/e4c7c548-c8af-40cc-9c22-a9f095c3c8f5/resourceceg...

I have to create a external location.....

Storage account: databricksdemo10121

Blob container: unity catalog

Folder (directroy): catalog

External location – it's a cloud path

Storage credentials

To enable fine grained access control...

Create a new credential ×

A storage credential represents an authentication and authorization mechanism for accessing data stored on your cloud tenant. [Learn more](#)

☒ Storage Credential ☐ Service Credential ⓘ

Credential Type*

Azure Managed Identity ▼

Credential name*

mycredential1

Access connector ID [Learn more](#) ⓘ

/subscriptions/e4c7c548-c8af-40cc-9c22-a9f095c3c8f5/resourceGroups/databricks_learning_demo/

User assigned managed identity ID (optional)

Comment

[> Advanced Options](#)

Cancel Create

For access connector_id → go to access connector for databricks → then copy resource Id

Then create external location:

[External Locations](#) >

Create a new external location

An external location allows you to access your data stored in cloud storage (e.g. Azure Data Lake Storage). You will need the cloud storage path and a paired credential (e.g. managed identity) which gives access to that path [Learn more](#)

External location name*

my_external_loc1

Storage type*

Azure Data Lake Storage

URL* ⓘ

Enter the bucket path that you want to use as the external location. Note: This must be an ADLS Gen2 storage account with a hierarchical namespace

abfss://unitycatalog@databricksdemo10121.dfs.core.windows.net/catalog

Storage credential* [Learn more](#)

Provide a storage credential capable of accessing the URL

mycredential1 (Managed Identity)

Connector Id: /subscriptions/e4c7c548-c8af-40cc-9c22-a9f095c3c8f5/resourceGroups/databricks_learning_demo/provide...
User Assigned Managed Identity Id:

Comment

|

Advanced Options

☐ Limit to read-only use

☐ Fallback mode ⓘ

We can create external location through code and manually as well.

```
%sql
```

```
CREATE EXTERNAL LOCATION my_external_location  
URL 'abfss://container@storageaccount.dfs.core.windows.net/path/'  
WITH (STORAGE CREDENTIAL my_storage_credential)  
COMMENT 'This is our primary raw data bucket';
```

Once we create the external location then we need run below commands...

```
%sql
```



```
create catalog my_catalog
managed location
'abfss://unitycatalog@databricksdemo10121.dfs.core.windows.net/catalog'
```

```
%sql
use catalog my_catalog;
```

```
%sql
create table student
(
    id int,
    name string
);
```

```
%sql
insert into student values (1,'Supriya'),(2,'naresh');
```

The screenshot displays the Databricks Catalog Explorer interface. On the left, a sidebar shows the 'Catalog' tree with 'my_catalog' expanded, revealing a 'default' schema containing a 'student' table. The main panel shows the 'student' table details under the 'Details' tab. The 'About this table' section provides the following information:

Property	Value
Created at	Aug 26, 2026, 09:10 AM
Created by	nareshnani2112.azure@gmail.com
Delta runtime properties kv pairs	(empty)
Storage location	abfss://unitycatalog@databricksdemo10121.dfs.core.windows.net/catalog/_unitystorage/catalogs/25a1ce3b-d737-497a-b456-46ead8a470aa/tables/f43bf9ed-480a-4152-889c-a107299cdf2b
Table ID	f43bf9ed-480a-4152-889c-a107299cdf2b
Type	MANAGED

We can see the storage location that is pointing to our storage location.

Creating External Locations in Databricks Using SQL

To create or add a storage path at the metastore level in Databricks, you must do it in the Databricks Account Console (not inside an individual workspace notebook)

[Accounts.databricks.com](https://accounts.databricks.com)

I want to create a schema level:

Create a folder with name schema in storage account.

```
%sql
CREATE EXTERNAL LOCATION my_external_location
URL 'abfss://unitycatalog@databricksdemo10121.dfs.core.windows.net/schema'
WITH (STORAGE CREDENTIAL my_storage_credential)
COMMENT 'This is our primary raw data bucket via schema level';
```

```
%sql
create schema <catalog>.<new_schema_name>
managed location
'abfss://unitycatalog@databricksdemo10121.dfs.core.windows.net/catalog'
```

Managed vs External Tables in Databricks

External Tables:

First we need to create a folder in storage account. Then we need to create a external location. Then create command with location.

```
%sql
CREATE EXTERNAL LOCATION my_external_location
URL 'abfss://unitycatalog@databricksdemo10121.dfs.core.windows.net/External'
WITH (STORAGE CREDENTIAL my_storage_credential)
COMMENT 'This is our primary raw data bucket to create external location';
```

```
%sql
create table if not exists students
(
  Id int,
  Name string,
  Score Float
)
using csv
location 'abfss://unitycatalog@databricksdemo10121.dfs.core.windows.net/External'
```

```
%sql
describe extended student;
```

```
%sql
insert into student values (1,'naresh',23);
```

- Always follow 3 level namespace... catalog/schema/table.
- Delta and Iceberg formats are supported for managed table.
- The role of managed resource group is : the resource group holds the resources which databricks creates in backed for example your vm's your data etc.

Week16:

S

All-Purpose Compute vs. Other Compute Types

Feature	All-Purpose Compute	Job Compute	Serverless Compute	SQL Warehouses
Primary Use	Ad-hoc coding, EDA, ML model building	Automated production workflows & pipelines	Fast notebook execution, serverless pipelines	Business intelligence tools and SQL queries
Lifecycle	Stays on until manual stop or inactivity timeout	Spins up automatically for a task and terminates instantly upon completion	Instant provisioning on-demand, manages infrastructure automatically	Auto-scales or stops based on SQL query activity
Languages	Python, SQL, Scala, R	Python, SQL, Scala, R	Python, SQL, etc.	SQL Only
Cost (DBU)	Highest per-DBU rate (billed for total uptime)	Lowest classic rate (only billed for execution time)	Medium-high rate (strictly billed for execution time)	Optimized per-DBU rate for BI tools

Job Compute:

Click on Existing notebook → Schedule →

New schedule

Job name*

Schedule

Every

Compute* ⓘ

Performance optimization ⓘ

☐ Performance optimized

More options

Under compute section: We have option “**Add new job cluster**”

[Compute](#) > [New compute](#) > Simple form: ON ▾

Create new compute

General

Compute name

My new compute resource

Policy

Unrestricted ▾

Performance

☐ Machine learning ⓘ

Databricks runtime

15.4 LTS Scala 2.12, Spark 3.5.0 ▾ ☒ Photon acceleration ⓘ

Worker type

rd-fleet.xlarge 32 GB Memory, 4 Cores ▾

Min Max

1 2 ☐ Single node

☒ Enable autoscaling ⓘ

Advanced performance ▾

Tags

Add tags

Key

Value

Add

> Automatically added tags

Advanced >

⌚

Create Cancel

3–6 DBU/h (Est.)

Policies:

Custom policies can be applied to two primary types of compute in Databricks: **All-Purpose Compute** and **Job Compute**.

Ex:

```
{  
  "cluster_type": {  
    "type": "fixed",  
    "value": "job",  
    "hidden": true  
  }  
}
```

Permissions:

When assigning access control lists (ACLs) to a custom Compute Policy.

Pools:

Cluster pools in Databricks are sets of idle, pre-configured virtual machine (VM) instances that allow clusters to start and scale up almost instantly

Databricks pools use on-demand instances for maximum reliability and strict deadlines, while spot instances (preemptible VMs) offer deep cost savings at the risk of sudden termination.

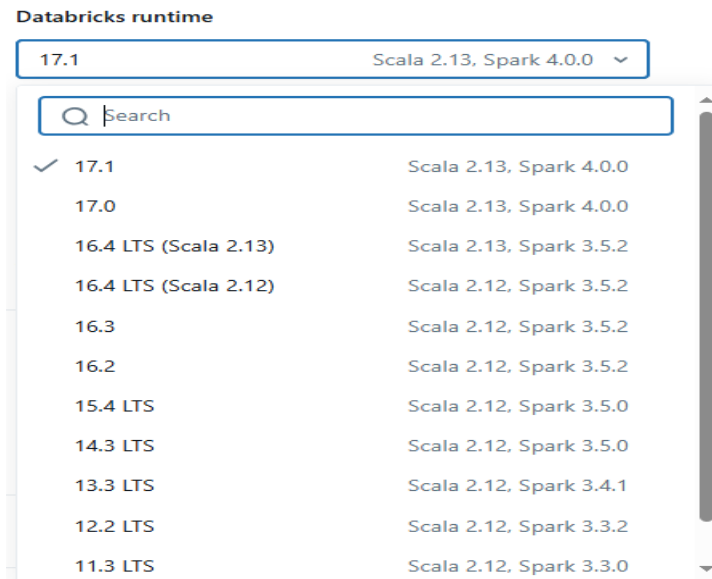
Key Differences

- **On-Demand Instances:**
 - Cost: Standard, full-price cloud provider rates.
 - Reliability: Guaranteed availability; instances will not be reclaimed by the cloud provider.
 - Use Case: Best for driver nodes, short jobs with strict execution time requirements, and mission-critical production pipelines
- **Spot Instances (Spot VMs):**
 - Cost: Up to 70-90% cheaper than on-demand pricing.
 - Reliability: Low; the cloud provider can reclaim these resources with little to no notice (**eviction**) when capacity is needed elsewhere.

- Use Case: Best for worker nodes in fault-tolerant, batch processing, development, or non-time-sensitive workloads

Once pool create chesaka aa pool ni create compute lo → Node Type (manam create chesina pool ni node type lo use chesukovachu).....

Databricks run time version:



This adds many features including....

Delta lake

Installed java,python,Scala and R libraries

Ubuntu and its system libraries

GPU libraries.

Delta Lake Basics - Managed Tables & Schema Definition

- ✚ Create resource group in central India region
- ✚ Create a storage account with name : **ttstorageaccount10123**
- ✚ Go to storage account → Access Control (IAM) → add role assignment → give permission (storage blob data contributor) → Select member → databricks connector name → close.

At databricks end:

Create secret creds:

Create a new credential

A storage credential represents an authentication and authorization mechanism for accessing data stored on your cloud tenant. [Learn more](#)

☒ Storage Credential ☐ Service Credential

Credential Type*

Azure Managed Identity

Credential name*

Access connector ID [Learn more](#)

User assigned managed identity ID (optional)

Comment

> Advanced Options

Cancel Create

Create a external location:

abfss://<container_name>@<storage_account_name>dfs.core.windows.net/<path>

abfss://unitycatalog@ttstorageaccount10123.dfs.core.windows.net/catalog

```
%sql
create catalog deltalake_catalog
managed location
'abfss://unitycatalog@ttstorageaccount10123.dfs.core.windows.net/catalog';
```



```
%sql
use catalog deltalake_catalog;
```

```
%sql
create or replace table orders_managed
(
  order_id bigint,
  sku string,
  product_name string,
  product_category string,
  qty int,
  unit_price decimal(10,2)
)
using delta;
```

Showing all 4 items

☐	Name	Last modified	Access tier	Blob type	Size
☐	 [-]				
☐	 __tmp_path_dir	8/31/2026, 10:10:16 PM			
☐	 __staged_commits	8/31/2026, 10:10:16 PM			
☐	 00000000000000000000.crc	8/31/2026, 10:10:17 PM	Hot (Inferred)	Block blob	2.83 KiB
☐	 00000000000000000000.json	8/31/2026, 10:10:16 PM	Hot (Inferred)	Block blob	2.19 KiB

Once the table creation was done we are able to view a json file.

Now insert few records:

```
%sql
insert into orders_managed values
(1, 'A101', 'Iphone charger', 'Electronics', 2, 19.99),
(2, 'A102', 'Headphones', 'Electronics', 1, 5.99),
(3, 'A103', 'HDMI cable', 'Electronics', 3, 9.99);
```

☐	Name	Last modified	Access tier
☐	 [.]		
☐	 __tmp_path_dir	8/31/2026, 10:10:16 PM	
☐	 _autostats	8/31/2026, 10:15:13 PM	
☐	 _staged_commits	8/31/2026, 10:10:16 PM	
☐	 00000000000000000000000000000000.crc	8/31/2026, 10:10:17 PM	Hot (Inferred)
☐	 00000000000000000000000000000000.json	8/31/2026, 10:10:16 PM	Hot (Inferred)
☐	 00000000000000000000000000000001.crc	8/31/2026, 10:15:12 PM	Hot (Inferred)
☐	 00000000000000000000000000000001.json	8/31/2026, 10:15:12 PM	Hot (Inferred)

We have another json file for inserting records....

Showing all 2 items

☐	Name	Last modified	Access tier
☐	 [.]		
☐	 _delta_log	8/31/2026, 10:10:16 PM	
☐	 part-00000-8721bedd-c0b6-4a94-99d3-e6ec796c6d1d.c000.zstd.parquet	8/31/2026, 10:15:11 PM	Hot (Inferred)

We have parquet files for actual data storing....

If we want to try to read .json file in databricks it is not possible.

here we can't see what inside the particular location(.json file) with help of spark.read or dbutils.

```
display(dbutils.fs.ls("abfss://unitycatalog@ttstorageaccount1023.dfs.core.windows.net/catalog/__unitystorage"))
```

```
ExecutionError: (com.databricks.common.client.DatabricksServiceHttpClientException)
INVALID_PARAMETER_VALUE: Input path url
'abfss://unitycatalog@ttstorageaccount1023.dfs.core.windows.net/catalog/__unitystorag
e' overlaps with managed storage within 'PathAuthzRequest' call.
```

```
+ Code + Text + Genie Code

Last execution failed 7

1 display(dbutils.fs.ls("abfss://unitycatalog@ttstorageaccount1023.dfs.core.windows.net/catalog/___unitystorage"))

> See performance \(1\) Optimize

> ExecutionError: (com.databricks.common.client.DatabricksServiceHttpClientException) INVALID_PARAMETER_VALUE: Input path url 'abfss://unitycatalog@ttstorageaccount1023.dfs.core.windows.net/catalog/___unitystorage' overlaps with managed storage within 'PathAuthzRequest' call. .

...

Diagnose error Debug Genie Code Quick Fix: OFF
```

```
Last execution failed 8

1 log_file_path = "https://ttstorageaccount1023.blob.core.windows.net/unitycatalog/catalog/___unitystorage/catalogs/e72408e2-d9df-4415-a436-51ca6bb03e1d/tables/a3f9716f-f644-4720-9234-48cb58038cab/_delta_log/00000000000000000001.json"
2
3 df_log=spark.read.json(log_file_path)
4 display(df_log)

> [FAILED_READ_FILE.UNSUPPORTED_FILE_SYSTEM] Error while reading file https://ttstorageaccount1023.blob.core.windows.net/unitycatalog/catalog/___unitystorage/catalogs/e72408e2-d9df-4415-a436-51ca6bb03e1d/tables/a3f9716f-f644-4720-9234-48cb58038cab/_delta_log/00000000000000000001.json. The file system org.apache.hadoop.fs.http.HttpsFileSystem hasn't implemented listStatus. SQLSTATE: KD001
```

Volumes:

```
%sql
create volume if not exists deltalake_catalog.default.delta_volume1;
```

```
# I want to create a dir
dbutils.fs.mkdirs("/Volumes/deltalake_catalog/default/delta_volume1/orders_data1")
```

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
schema = StructType([
    StructField("employee_id", IntegerType(), False),
    StructField("name", StringType(), True),
    StructField("department", StringType(), True)
])

data = [
    (101, "Alice", "Engineering"),
    (102, "Bob", "Data Science"),
    (103, "Charlie", "Product")
]

df = spark.createDataFrame(data, schema)
df.show()
df.printSchema()
```

```
volume_path='/Volumes/deltalake_catalog/default/delta_volume1/orders_data1'  
df.write.format("delta").mode("overwrite").save(volume_path)
```

```
%sql  
SELECT * from delta.`/Volumes/deltalake_catalog/default/delta_volume1/orders_data1`
```

How to read JSON files:

```
%fs ls /Volumes/deltalake_catalog/default/delta_volume1/orders_data1
```

```
%fs ls /Volumes/deltalake_catalog/default/delta_volume1/orders_data1/_delta_log
```

```
log_file_path="dbfs:/Volumes/deltalake_catalog/default/delta_volume1/orders_data1/_delta_log/00000000000000000000.json"  
df_log=spark.read.json(log_file_path)  
display(df_log)
```

External Tables:

We need to create a path in storage account.

```
%sql  
create or replace table orders_external  
(  
    order_id bigint,  
    sku string,  
    product_name string,  
    product_category string,  
    qty int,  
    unit_price decimal(10,2)  
)  
location 'abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders'
```

```
%sql
insert into orders_external values
(1,'A101','Iphone charger','Electronics',2,19.99),
(2,'A102','Headphones','Electronics',1,5.99),
(3,'A103','HDMI cable','Electronics',3,9.99);
```

```
display(dbutils.fs.ls('abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/
orders/_delta_log'))
```

```
%sql
select * from
json.`abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders/_delta_lo
g/00000000000000000001.json`
```

```
%sql
describe history orders_external;
```

operation

WRITE

CREATE OR REPLACE TABLE

```
%sql
insert overwrite orders_external values
(4,'A101','Iphone pro max 18','Electronics',2,19.99);
```

```
%sql
describe history orders_external;

-- In metadata only.. the physical files are still be there
-- numFiles: "1"
-- numRemovedFiles: "1"
```

%md

so before overwrite We have 1 parquet file and 2 json files (1 for creation of table and 2 for insert command)

then when we did overwrite we can still able above view files and 1 parquet file (for overwrite operation) and one more json file.(total 3 json files)

```
display(dbutils.fs.ls('abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders/_delta_log'))
```

%sql

```
select * from
```

```
json.`abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders/_delta_log/00000000000000000002.json`
```

%md

```
json1 -- add parquet1
```

```
json2 --> add parquet2  
          remove parquet1
```

```
add parquet2
```

NEW DEMO1:

%sql

```
create or replace table orders_external_01
```

```
(
```

```
  order_id bigint,  
  sku string,  
  product_name string,  
  product_category string,  
  qty int,  
  unit_price decimal(10,2)
```

```
)
```

```
location 'abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders'
```

```
%sql
insert into orders_external_01 values
(1, 'A101', 'Iphone charger', 'Electronics', 2, 19.99),
(2, 'A102', 'Headphones', 'Electronics', 1, 5.99),
(3, 'A103', 'HDMI cable', 'Electronics', 3, 9.99);
```

```
%sql
insert into orders_external_01 values
(4, 'A101', 'Iphone charger', 'Electronics', 2, 19.99),
(5, 'A102', 'Headphones', 'Electronics', 1, 5.99);
```

```
%sql
describe history orders_external_01;
```

operation

WRITE

WRITE

CREATE OR REPLACE TABLE

```
%md
lets delete a order id =3 (the 3 is reides in the first parquet file)
```

```
%sql
delete from orders_external_01 where order_id=3;
```

```
%sql
describe history orders_external_01;
```

operation

OPTIMIZE

DELETE

WRITE

WRITE

CREATE OR REPLACE TABLE

Part-001.parquet → 1,2,3

Part -002.parquet → 4,5

Without Deletion vector:

Part-001.parquet → before 1,2,3 AFTER part-003.parquet → 1,2

And marks as

Part-001.parquet → removed

Part-003.parquet → added.

With Deletion vector:

Part-001.parquet → before 1,2,3 --→ Row 3 deleted

So the physical parquet file can still contain 3

Lets suppose 10 million records in a file:

Deleted 1 record

Parquet 1 → 10 million records

Parquet 2 → 10 million records -1 record

In delta log it says:

Remove parquet1

Add parquet2


```
display(dbutils.fs.ls('abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders/_delta_log'))
```

```
%sql
select * from
json.`abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders/_delta_log/00000000000000000003.json`
```

under remove file:

path: "part-00000-33701765-a037-433b-9de1-6c1089fdc5df.c000.snappy.parquet"

size: 2022

stats: "{\"numRecords\":3} (the file whcih we inserted first set of records)

ADD:

path: "part-00000-33701765-a037-433b-9de1-6c1089fdc5df.c000.snappy.parquet"

size: 2022

stats: "{\"numRecords\":3}, along with deletion vector

so we are removing same file and adding the same file.....

for example

10 million rcords -- parquet1

you are deleting 1 record

a deletion vector file is created -- remove id 3

no new parquet file will be created

In json:

remove parquet1

add parquet1 along with deletion vector...

lets perform 2 deletes

```
%sql
create or replace table orders_external_01
(
    order_id bigint,
    sku string,
    product_name string,
    product_category string,
    qty int,
    unit_price decimal(10,2)
)
location 'abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders'
```

```
%sql
insert into orders_external_01 values
(1, 'A101', 'Iphone charger', 'Electronics', 2, 19.99),
(2, 'A102', 'Headphones', 'Electronics', 1, 5.99),
(3, 'A103', 'HDMI cable', 'Electronics', 3, 9.99);
```

```
%sql
insert into orders_external_01 values
(4, 'A101', 'Iphone charger', 'Electronics', 2, 19.99),
(5, 'A102', 'Headphones', 'Electronics', 1, 5.99);
```

```
%sql
delete from orders_external_01 where order_id IN (1,5);
```

```

file 1 --- 1,2,3
file 2 -- 4,5

deleted 1,5

add two deletion vector

deletionvector1 -- id 1
deletionvector1 -- id 5

remove file1
remove file2
add file1 with deletion vector1
add file2 with deletion vector2

```

A deletion vector is associated with a specific data file and row positions affected by the deletion.

If your delete touches 100 different parquet files, Delta can maintain deletion information for those 100 files without necessarily rewriting all 100 files immediately...

Update:

Part1.parquet → 1,2,3

Par2.parquet → 4,5

```

%sql
update orders
set status='Shipped' where order_id=3;

```

part1.parquet

ID 1

ID 2

ID 3 → marked deleted by deletion vector.

Part2.parquet → it it not touched it contains 4 and 5 as it is....

Part3.parquet → new file will be created with only updated record only (not included 1,2)...

DELETE: Deletion vector can mark the old row as deleted without creating replacement data.

UPDATE: the old row can be marked as deleted using a DV, **while the updated version is written to the new data file...**

WEEK17:

Computing the Latest State in Delta Lake

How delta lake computes the latest state:

- ➔ *Created a table (000.json)*
- ➔ *Inserting 5 records in a single statement*

File1.parquet

001.json (add file1.parquet

-> *updating one of the record*

Dv file

File2.parquet (only updated record information)

002.json (remove 1.parquet , add 1.parquet with dv , add file2.parquet)

Select * from table;

000.json

001.json

002.json

Add file1.parquet **X (it is nullified)**

Remove file1.parquet **X(it is nullified)**

Add file1.parquet with DV

Add file2.parquet

When We hit select * from table we will get the reponse from only above 2 files. (Bold lines)

Lets say you perform a overwrite.

File3.parquet

003.json (remove file1.parquet with DV, remove file2.parquet, add file3.parquet)

Add file1.parquet **X(it is nullified)**

Remove file1.parquet **X(it is nullified)**

Add file1.parquet with DV X(it is nullified)

Add file2.parquet X(it is nullified)



Remove file1.parquet with DV **X(it is nullified)**

Remove file2.parquet **X(it is nullified)**

Add file3.parquets

*I hit select * we will get the result from file3.parquet only...*

Try with below example:

```
%sql
create or replace table orders_external_01
(
    order_id bigint,
    sku string,
    product_name string,
    product_category string,
    qty int,
    unit_price decimal(10,2)
)
location 'abfss://externaldata@ttstorageaccount1023.dfs.core.windows.net/orders'
```

Delta Lake - Time Travel & Table Restore

```
%sql
describe history orders_external_01;
```

> [See performance \(1\)](#) ✓ Checked for i

Table ▼ + Q 5

	version	timestamp	userId	userName	operation
1	6	2026-09-05T12:52:10.000+00:00	11135336969592...	nareshboda112@gmail.co...	WRITE
2	5	2026-09-05T12:51:48.000+00:00	11135336969592...	nareshboda112@gmail.co...	OPTIMIZE
3	4	2026-09-05T12:51:46.000+00:00	11135336969592...	nareshboda112@gmail.co...	DELETE
4	3	2026-09-05T12:51:08.000+00:00	11135336969592...	nareshboda112@gmail.co...	OPTIMIZE
5	2	2026-09-05T12:51:05.000+00:00	11135336969592...	nareshboda112@gmail.co...	UPDATE
6	1	2026-09-05T12:47:43.000+00:00	11135336969592...	nareshboda112@gmail.co...	WRITE
7	0	2026-09-05T12:38:00.000+00:00	11135336969592...	nareshboda112@gmail.co...	CREATE OR REPLACE TABLE

```
%sql
select * from orders_external_01 VERSION AS OF 6;
```

```
%sql
select * from orders_external_01@v2;
```

```
%sql
select * from orders_external_01 timestamp as of '2026-09-05T12:51:46.000+00:00';
```

Using Python style with Dataframes:

```
df_latest=spark.read.table("orders_external_01")
df_latest.show()
```

```
df_latest=spark.read.option("versionAsOf",2).table("orders_external_01")
df_latest.show()
```

```
df_latest=spark.read.format("delta") \
    .option("timestampAsOf", '2026-09-05T12:51:46.000+00:00') \
    .table("orders_external_01")
df_latest.show()
```

If I want to restore the table at a particular version:

```
%sql
RESTORE table orders_external_01 to version as of 1;
```

Once we restore the table we will go to at a particular version:

```
%sql
select * from orders_external_01 version as of 6;
```

to identify the current active pointing file details are:

SELECT distinct _metadata.file_path from <table_name>

To view history:

DESCRIBE HISTORY <catalog> <schema><table_name>

VACUUM & Table Optimization: (remove parquet files)

- ✚ Vacuum removes data files that are no longer referenced by a delta table version in the last 7 days.
- ✚ vacuum command is a maintenance command.
- ✚ Delta.deletedFileRetentionDuration=30days (for the longer duration if we want to change from 7 to 30 days)
- ✚ Alter table <table_name> SET tblproperties (delta.deletedFileRetentionDuration = '30 Days');
- ✚ By default databricks protect you with a 7-day limit increasing this safety limit to 30 days.

Purpose: permanently deletes unreferenced data files from cloud storage.

Benefit: reduces storage costs and cleanup deleted/ updated data “trash”

Vacuum <table_name>

Test first:

Vacuum table_name DRY RUN (preview upto 1000 files)

Risk: deleted history cannot be recovered via “TIMESTAMP AS OF “

```
%sql
create or replace table order_ext_01
(
    order_id bigint,
    sku string,
    product_name string,
    product_category string,
    qty int,
    unit_price decimal(10,2)
)
TBLPROPERTIES (
    delta.autoOptimize.optimizeWrite=false,
    delta.autoOptimize.autoCompact=false
)
LOCATION 'abfss://externaldata@ttstorageaccount001.dfs.core.windows.net/orders'
```

```
%sql
show TBLPROPERTIES orders_ext_01;
```

```
%sql
vacuum order_ext_01 retain 0 hours dry run;
```

Managing Delta Table Commits

Its all about reading json files for effective reading

=====

To keep deletion vectors from piling up again in the future, set up a scheduled production maintenance job (such as a nightly workflow) that handles the automated cleanup: [1]

1. **REORG TABLE my_table APPLY (PURGE);** (Removes the vectors and rewrites the active data).
2. **VACUUM my_table;** (Safely cleans out the historic cloud file fragments)

what If I run optimize command:

Running the OPTIMIZE command will also get rid of your deletion vectors, but it does it as part of a larger file-compaction process.

OPTIMIZE vs. REORG TABLE ... APPLY (PURGE)

While both commands will eliminate deletion vectors, they have different focus areas:

Feature	OPTIMIZE	REORG ... APPLY (PURGE)
Primary Goal	Merges small files together for faster reads.	Removes deletion vectors to purge deleted data.
Target Scope	Scans the entire table/partition to find small files.	Target-scans only the files that have deletion vectors.
Performance Impact	Heavier/Slower (does a lot of data moving and restructuring).	Lighter/Faster (only fixes files with soft-deleted rows).

*Run **REORG** if your files are already a good size, but you just want to quickly clear out deletion vectors (e.g., after a daily GDPR deletion or record purge).*

Run **OPTIMIZE** if your deletion vectors are piling up and your table is suffering from the "small file problem" due to frequent streaming or micro-batch writes.

Example:

Imagine you have a table tracking sales data. It currently contains 3 files:

- File A (10 MB): A tiny file containing recent transactions.
- File B (10 MB): Another tiny file containing recent transactions.
- File C (1,000 MB): A perfectly sized, healthy 1 GB production file.

You run a delete command to remove a single bad record from File C. Because deletion vectors are enabled, Databricks does not rewrite the heavy 1 GB file. Instead, it leaves File C untouched and creates a tiny Deletion Vector (DV) file right next to it.

Now, your storage looks like this:

- File A (10 MB)
- File B (10 MB)
- File C (1,000 MB) +  Deletion Vector

Scenario 1: If you run REORG TABLE ... APPLY (PURGE)

REORG is a laser-focused sniper. It *only* cares about removing deletion vectors.

1. It looks at the table and says: *"File A and File B are clean. I will ignore them."*
2. It looks at File C and says: *"This has a deletion vector. I need to fix it."*
3. It reads File C, drops the 1 deleted record, and writes a fresh File C_new (1,000 MB).
4. The deletion vector is gone.

The Result: You have File A (10MB), File B (10MB), and File C_new (1000MB). Your deletion vector is successfully removed, but your small file problem (File A and File B) remains untouched.

Scenario 2: If you run OPTIMIZE

OPTIMIZE is a heavy-duty cleanup crew. It clears deletion vectors *and* reorganizes the whole table layout.

1. It looks at File C and says: *"This has a deletion vector. I will rewrite it to purge the deleted row."*
2. It looks at File A and File B and says: *"These files are way too small. I need to merge them together to improve read performance."*
3. It combines File A and File B into a brand new File AB (20 MB).
4. It writes a fresh File C_new (1,000 MB).
5. The deletion vector is gone.

The Result: You have File AB (20MB) and File C_new (1000MB). Your deletion vector is gone, and your small files have been consolidated.

- **If your pipeline creates lots of tiny files (e.g., streaming every few minutes), run OPTIMIZE. It cleans up the vectors and fixes the small file layout simultaneously.**
- **If your files are already large and healthy, but you just did a clean-up delete, run REORG. It saves massive cluster computing costs by skipping files that do not have vectors.**

Under 128 MB? You have a small file problem. Run OPTIMIZE to merge them and drop the vectors at the same time.

Close to 1 GB? Your file sizes are perfectly healthy. Run `REORG TABLE ... APPLY (PURGE)` to target only the deletion vectors and save cluster costs.

Handling Deletion vectors:

Step1: Create external table

Create or replace table <table_name>

```
(
Order_id bigint,
Sku string,
Product_name string,
Product_category string,
Qty int,
Unit_price decimal(10,2)
)
TBLPROPERTIES (
delta.autoOptimize.optimizeWrite=false,
delta.autoOptimize.autoCompact=false,
)
Location 'abfss://<external_loc_path>'
```

Step2: Insert 5 records

Step3: DELETE FROM <TABLE> WHERE ORDER_ID=4;

It will create deletion vector in storage

Step4: Describe history <table_name>

SO We can see 3 operations Create or replace , Write, Delete.. after that we optimize command won't run because when we create a table we give under TBLPROPERTIES.

Step5: REORG TABLE <table_name> APPLY (PURGE)

Check metrics

Step6: Describe history <table_name>

APPLY (PURGE) only rewrites files that contain soft deleted data.

The Two Steps of Deletion

1. The DELETE Command (Soft Delete)

- When you run *DELETE FROM table WHERE id = 123*, Databricks does not touch your data files.
- It just writes a hidden note (a deletion vector) that says: "Hide record 123 from view."
- The record is hidden from queries, but it is **still physically sitting** inside your storage file.

2. The PURGE Command (Hard Physical Delete)

- When you run *REORG TABLE table APPLY (PURGE)*, Databricks reads that hidden note.
- It takes the file, creates a new copy of it, and **completely leaves out** record 123.
- Once this command finishes, **record 123 is physically gone** from that active file.

When Bob data deleted.

What happens under the hood: Databricks scans the table, finds the file containing Bob's data, and writes a **brand new file** containing only Alice and Charlie.

The Catch: The old file (the one containing Alice, Bob, and Charlie) is still kept in your cloud storage for "Time Travel" purposes, just in case you need to roll back the table to how it looked yesterday.

Final Step:

Apply VACUUM (The Final Clean)

To ensure Bob's data is 100% gone from the cloud storage disks and cannot be recovered using Time Travel, you must remove the old files.

APPLY(UPGRADE) may rewrite: *may rewrite all the files.*

REORG TABLE is an idempotent operation, meaning that if we run it twice on the same dataset, the second run has no effect.

After running apply (purge), the soft deleted data may still exists in the old files. You can run **VACUUM** to physically delete the old files.

UPSERT / MERGE:

Upsert = update + insert

Ex:

Incoming batch of data: (Source)

3,

4,.....

5,.....

Orders table – (Target):

1,..... (order_id,some columns)

2,.....

3,.....

Order id 3 should be updated

Order id 4 and 5 should be inserted.

Pyspark Code:

```
from delta.tables import DeltaTable
```

1. Reference the target Delta table using its path (Volume location)

```
target_table = DeltaTable.forPath(spark,  
"/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1")
```

2. Execute the merge operation

```
target_table.alias("target") \  
  .merge(  
    source=incoming_snapshot.alias("src"),  
    condition="target.order_id = src.order_id"  
  ) \  
  .whenMatchedUpdateAll() \  
  .whenNotMatchedInsertAll() \  
  .whenNotMatchedBySourceDelete() \  
  .execute()
```

// refer upsert notebook in databricks

```
%sql  
USE catalog deltalake_demo;
```

```
%sql  
CREATE VOLUME IF NOT EXISTS deltalake_demo.default.deltalake_demo;
```

```
dbutils.fs.mkdirs("/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1")
```

UPSERT DATA SETUP:


```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType, LongType
```

```
# 1. Define the schema using StructType
```

```
schema = StructType([
    StructField("order_id", LongType(), False),
    StructField("sku", StringType(), False),
    StructField("product_name", StringType(), True),
    StructField("product_category", StringType(), True),
    StructField("qty", IntegerType(), True),
    StructField("unit_price", DoubleType(), True)
])
```

```
# 2. Define the sample data
```

```
data = [
    (1, "SKU-1023", "Wireless Mouse", "Electronics", 2, 25.50),
    (2, "SKU-4051", "Mouse Pad", "Electronics", 1, 9.99),
    (3, "SKU-8892", "Running Shoes", "Apparel", 1, 89.95),
    (4, "SKU-3112", "Coffee Mug", "Kitchen", 4, 12.00),
    (5, "SKU-1023", "Wireless Mouse", "Electronics", 1, 25.50)
]
```

```
# 3. Create the DataFrame
```

```
df = spark.createDataFrame(data, schema=schema)
```

```
# Show the DataFrame and its schema
```

```
df.show()
df.printSchema()
```

```
<> + Code + Text Genie Code
Python
df.show()
```

order_id	sku	product_name	product_category	qty	unit_price
1	SKU-1023	Wireless Mouse	Electronics	2	25.5
2	SKU-4051	Mouse Pad	Electronics	1	9.99
3	SKU-8892	Running Shoes	Apparel	1	89.95
4	SKU-3112	Coffee Mug	Kitchen	4	12.0
5	SKU-1023	Wireless Mouse	Electronics	1	25.5

```
volume_path="/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1"
df.write.format("delta").mode("overwrite").save(volume_path)
```

```
%sql
DESCRIBE HISTORY DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1`
```

Operation:

WRITE

```
%md
lets assume below table as target batch
```

```
%sql
SELECT * FROM DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1`
```

----->>> <<<-----

```
%md
Assume below data is source data
```

```
%sql
CREATE OR REPLACE TEMP VIEW incoming_batch AS
SELECT * FROM VALUES
    (2, "SKU-4051", "Yoga Mat-Updated", "Electronics", 1, 9.99),
    (99, "SKU-4099", "Inserted", "Electronics", 1, 9.99),
    (98, "SKU-40914", "Inserted", "Electronics", 1, 9.99)
AS t(order_id,sku,product_name,product_category,qty,unit_price);
```

MERGE command:

```
%sql
MERGE INTO DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1` as
target
USING incoming_batch as source
on target.order_id=source.order_id
WHEN MATCHED THEN UPDATE SET
target.sku=source.sku,
target.product_name=source.product_name,
target.product_category=source.product_category,
target.qty=source.qty,
target.unit_price=source.unit_price
WHEN NOT MATCHED THEN INSERT
(order_id,sku,product_name,product_category,qty,unit_price)
VALUES
(source.order_id,source.sku,source.product_name,source.product_category,source.qty,so
urce.unit_price);
```

```
%sql
SELECT * FROM DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1`
```

```
%sql
DESCRIBE HISTORY DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1`;
```

operation

OPTIMIZE

MERGE

WRITE

```
%md
LETS ADD FEW MORE RECORDS
```

```
%sql
CREATE OR REPLACE TEMP VIEW incoming_flag AS
SELECT * FROM VALUES
    (2, "SKU-4051", "Yoga Mat-re-deleted", "Electronics", 1, 9.99,False),
    (99, "SKU-4099", "Inserted", "Electronics", 1, 9.99,True)
AS t(order_id,sku,product_name,product_category,qty,unit_price,is_deleted);
```

```
%sql
MERGE INTO DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1` as
target
USING incoming_flag as source
on target.order_id=source.order_id
WHEN MATCHED AND source.is_deleted=true THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

- **Delete Condition:** If order_id matches **and** is_deleted is true → **Removes** the row from the target.
- **Update Condition:** If order_id matches (but isn't deleted) → **Overwrites** all target columns with source data.
- **Insert Condition:** If order_id is brand new → **Inserts** the entire new row into the target.

```
%sql
-- snapshot (only these rows should remain after Merge)
CREATE OR REPLACE TEMP VIEW incoming_snapshot AS
SELECT * FROM VALUES
    (1, "SKU-1023", "Wireless Mouse", "Electronics", 2, 25.5),
    (2, "SKU-4051", "Yoga Mat-re-deleted", "Electronics", 1, 9.99)
AS t(order_id,sku,product_name,product_category,qty,unit_price);
```

```
%sql
MERGE INTO DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1` AS
target
USING incoming_snapshot as src
on target.order_id=src.order_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

```
%sql
SELECT * FROM DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1`
```

```
%sql
DESCRIBE HISTORY DELTA.`/Volumes/deltalake_demo/default/deltalake_demo/ordersdata1`;
```

operation

OPTIMIZE

MERGE

OPTIMIZE

MERGE

OPTIMIZE

MERGE

OPTIMIZE

MERGE

WRITE

Constraints / ACID / Optimistic Concurrency Control

```
%sql
```

```
Use catalog deltalake_catalog;
```

```
Create table orders_Ext
```

(
)

Location <>

Lets insert 5 records.....

Order_id → (NOT NULL)

Qty, unit_price > 0 (should be greater than 0)

%sql

ALTER TABLE orders_Ext

ALTER COLUMN order_id SET NOT NULL;

DESCRIBE HISTORY orders_ext;

Then we can view 3 operations → Create or Replace Table, Write, **CHANGE COLUMN**.

I want to add a check constraint.

%sql

ALTER TABLE orders_Ext

ADD CONSTRAINT chk_qty_price CHECK (qty>0 and unit_price >0);

DESCRIBE HISTORY orders_ext;

Then we can view 3 operations → Create or Replace Table, Write, **CHANGE COLUMN,ADD CONSTRAINT;**

ON THE FLY IT WILL CHECK THE EXISTING THE DATA AS WELL.if any of the data is not complied with constraint then it throw an error.

%sql

INSERT INTO orders_ext_01 VALUES (6,'SKU-701','Faulty Item','MISC',0,-10.00)

ERROR BECAUSE OF VIOLATION CONSTRAINT. It is...consistence.

%sql

Create a temp source table (simulate an incoming batch with one bad row and one record
Is fine) → *Atomicity*

CREATE OR REPLACE TEMP VIEW incoming AS

SELECT * FROM VALUES

(2,'SKU-701','Faulty Item','MISC',0,10.00),

(8,'SKU-701','Faulty Item','MISC',0,-10.00)

AS (order_id,sku,product_name,product_category,qty,unit_price);

%sql

MERGE INTO orders_Ext AS Target

USING incoming as src

ON target.order_id = src.order_id

WHEN MATCHED THEN UPDATE SET *

WHEN NOT MATCHED THEN INSERT *;

---< ERROR →

Update (it should created parquet file, deletion vector)

Insert (new parquet).

This will show the automatic behaviour.

Isolation:

- Pessimistic Concurrency control

We lock the row then perform the operations then unlock it. (Row Level Locking).. In Older data base we are using above approach..

Optimistic Concurrency Control

The Setup

Imagine a Delta table containing user profiles. The latest version of the table is **Version 10**. Both users start their work at exactly the same time.

Step 1: Read (Both users start)

- **User 1** starts a job to fix a typo: UPDATE profiles SET name = 'Alice Smith' WHERE id = 1
- **User 2** starts a job to update a status: UPDATE profiles SET status = 'Active' WHERE id = 1
- **Both users read Version 10** of the table metadata. They both identify that the data for id = 1 resides inside a file named part-001.parquet.

Step 2: Write (In isolation)

Because Delta data files are immutable, neither user modifies part-001.parquet directly. Instead, they copy the data, apply changes, and write new files.

- **User 1** writes a new file: part-002-temp.parquet (with the name update).
- **User 2** writes a new file: part-003-temp.parquet (with the status update).

Step 3: Commit (The race condition)

This is where OCC validates the transactions. **User 1 completes their write slightly faster** and attempts to commit first.

1. User 1 Commits:

- Delta Lake checks if anyone committed after Version 10. No one has.
- User 1 successfully commits. This change becomes **Version 11**.

- part-001.parquet is marked as logically deleted, and part-002-temp.parquet is officially added.

2. User 2 Commits (Seconds later):

- User 2 attempts to commit their change as the next version.
- Delta Lake validates the commit and says: *"Wait, Version 11 was just created by User 1 while you were working."*
- Delta Lake checks what User 1 changed. It sees User 1 modified part-001.parquet.
- Delta Lake checks what User 2 wants to modify. It sees User 2 *also* relied on the old part-001.parquet.

The Outcome

Because both users tried to modify the exact same underlying file (part-001.parquet) concurrently, Delta Lake detects a collision.

- **User 2's transaction fails** and throws a ConcurrentDeleteDeleteException.
- User 2's staged file (part-003-temp.parquet) is ignored and later cleaned up by the system.
- To succeed, User 2 must retry the operation. The retry will read the fresh **Version 11**, see Alice's new name, and cleanly apply the status update into a brand new Version 12.

Only **update,delete,merge** will get above scenario.....

Schema Enforcement & Schema Evolution:

Schema Enforcement – is the process of strictly maintaining a predefined schema. Rejecting any data that does not conform to it to ensure data quality and consistency.

It ensures that any data being written to a table exactly matches the pre-defined structure (schema) of that table.

If the data does not match, Delta Lake rejects the write entirely instead of inserting bad or misaligned data.

*Ex: If qty is **int** but we are getting as **string***

```
CREATE OR REPLACE TEMP VIEW incoming AS
```

```
SELECT * FROM VALUES
```

```
(2,'SKU-701','Faulty Item','MISC','two',10.00)
```

```
AS (order_id,sku,product_name,product_category,qty,unit_price);
```

```
INSERT INTO DELTA.`/volumnes/deltalake_catalog/default/delta_volume1/ordersdata`
```

We will get 'CAST_INVALID_INPUT' ERROR....

Ex2:

----- create a temp view with wrong daata type for qty (string 'two')

```
CREATE OR REPLACE TEMP VIEW incoming AS
```

```
SELECT * FROM VALUES
```

```
(2,'SKU-701','Faulty Item','MISC','2',10.00)
```

```
AS (order_id,sku,product_name,product_category,qty,unit_price);
```

----- *This insert should works fine.. bcz it is type casted.*

```
INSERT INTO DELTA.`/volumnes/deltalake_catalog/default/delta_volume1/ordersdata`
```

What if the value is 5.2 then insert operation works fine.. it took 5 as value...

What if the value is "2" then insert operation works fine.. it took 2 as value...

Ex3: Source has an extra column 'discount'

```
CREATE OR REPLACE TEMP VIEW extra_col_src AS
```

```
SELECT * FROM VALUES
```

```
(2,'SKU-701','Faulty Item','MISC',1,10.00,5.0)
```

```
AS (order_id,sku,product_name,product_category,qty,unit_price,discount);
```

```
INSERT INTO DELTA.`/volumnes/deltalake_catalog/default/delta_volume1/ordersdata`
```

```
SELECT * FROM extra_col_src; ---→ <ERROR>
```

I want to evolve the Schema ... **Here we are doing Manually.....**

```
ALTER TABLE
```

```
DELTA.`/volumnes/deltalake_catalog/default/delta_volume1/ordersdata`
```

```
ADD COLUMN (discount DOUBLE);
```

---- *Confirm Schema*

```
DESCRIBE DETAIL
```

```
DELTA.`/volumnes/deltalake_catalog/default/delta_volume1/ordersdata`
```

----- Now insert the row with extra column (works fine)..

```
INSERT INTO DELTA.`/volumnes/deltalake_catalog/default/delta_volume1/ordersdata`
```

```
SELECT order_id,sku,product_name,product_category,qty,unit_price,discount FROM  
extra_col_src;
```

Schema Evolution:

Create or replace temp view incoming_with_new_col1 as

```
SELECT * FROM VALUES
```

```
(13,'sku-101','Auto Evolve Item','Promo',2,199.00,5,'PROMO101')
```

```
AS t
```

```
(order_id,sku,product_name,product_category,qty,unit_price,discount,promotion_code);
```

```
%sql
```

```
MERGE WITH SCHEMA EVOLUTION INTO DELTA
```

```
.` /volumnes/deltalake_catalog/default/delta_volume1/ordersdata` AS target
```

```
USING incoming_with_newcol as src
```

```
ON target.order_id =src.order_id
```

```
WHEN MATCHED THEN UPDATE SET *
```

```
WHEN NOT MATCHED THEN INSERT *;
```

In Python Way:

Create dataframe with new column promotion_code with StructType and StructField.

```
Df.write \
```

```
.format("delta") \
```

```
.mode("append") \
```

```
.option("mergeSchema","true") \
```

```
.save('/volumnes/deltalake_catalog/default/delta_volume1/ordersdata')
```

When the target table qty type is 'int' and in my source I am getting qty is long type it will not work. To make it work we need to enable ...

```
%sql
```

```
Alter table DELTA>` /volumnes/ deltalake_catalog/default/delta_volume1/ordersdata'
```

```
SET tblproperties ('delta.enableTypeWidening'='true');
```

```
%sql
```

```
Show tblproperties DELTA.
```

```
`/volumnes/deltalake_catalog/default/delta_volume1/ordersdata'
```

Schema evolution –

the feature in Delta Lake that allows you to change a table's current schema to accommodate data that changes over time.

Instead of rejecting a write when new columns are introduced, schema evolution automatically adjusts the table's layout to seamlessly integrate the new structure.

is the flexible process of updating and adapting a dataset's schema to accommodate changes. Such as adding new column or modifying datatypes, without breaking existing data pipelines.

- Existing data files are **not rewritten**.
- Historical records simply display NULL values for any newly added columns when queried.
- Downstream queries and BI tools continue to work without breaking.

Supported Schema Changes

Delta Lake allows you to make several structural changes safely: [\[1\]](#)

- **Adding New Columns:** Adding columns to the end of the table or inside nested structures.
- **Changing Nullability:** Changing a column constraint from NOT NULL to NULL (allowing empty values).
- **Upcasting Data Types:** Safely widening a data type, such as changing an INT to a BIGINT or a FLOAT to a DOUBLE. [\[1, 2, 3, 4, 5\]](#)

Note: Demoting a data type (e.g., BIGINT to INT) or dropping/renaming a column usually requires explicit commands or a table rewrite to prevent data truncation or loss.

Adding new column -> `.option("mergeSchema", "true").`

Data type widening → **SET tblproperties ('delta.enableTypeWidening'='true');**

Rename a column or Delete a column:

In parquet file you have data also it contains the column names.

It created a new log file (metadata file).

Azure databricks supports column mapping for delta lake tables, which enables metadata-only changes to mark columns as deleted or renamed without rewriting data files.

Column mapping:

Physical file: qty

Logical name : quantity

Delta Lake Column Mapping decouples the physical names of columns in the underlying Parquet data files from their logical names in the table metadata. Historically, renaming a column required completely rewriting the underlying data files, which was incredibly slow and expensive. Column mapping turns renaming into a **metadata-only operation that takes milliseconds**

When you enable column mapping, Databricks assigns a unique internal identifier (an ID or a unique text string) to every column.

- **The Physical File:** The data in storage continues to use the old physical column name or a random alphanumeric string.
- **The Metadata Log:** The `_delta_log` records that the logical name has changed, mapping your new logical name to the original physical location.

The Query Engine: When you query the table using the new column name, Databricks seamlessly reads the mapped physical column in the background.

Step 1: Enable Column Mapping

Column mapping is **disabled** by default (`none`). You must turn it on before you can execute a metadata-only rename command.

For a New Table:

Use the `id` mode when creating a table from scratch.

sql

```
CREATE TABLE my_catalog.my_schema.employee_data (  
  emp_id INT,  
  emp_name STRING  
)  
USING DELTA  
TBLPROPERTIES ('delta.columnMapping.mode' = 'id');
```

For an Existing Table:

Use the `name` mode if the table already contains data. [\[1\]](#)

sql

```
ALTER TABLE my_catalog.my_schema.employee_data  
SET TBLPROPERTIES ('delta.columnMapping.mode' = 'name');  
Use code with caution.
```

Step 2: Run the Rename Command

Once enabled, you can alter the table to rename the column safely without executing any data rewrite.

sql

```
ALTER TABLE my_catalog.my_schema.employee_data  
RENAME COLUMN emp_name TO employee_full_name;
```

Step 1: You Create the Table

```
CREATE TABLE employee_table (emp_id INT, first_name STRING)
```

USING DELTA TBLPROPERTIES ('delta.columnMapping.mode' = 'id');

Behind the scenes:

Databricks writes a hidden map into the Delta Log (`_delta_log/000000.json`). It assigns internal physical IDs to your columns:

- `emp_id` → Map to **ID 1**
- `first_name` → Map to **ID 2**

Step 2: You Insert Data

You insert your first row:

sql

```
INSERT INTO employee_table VALUES (101, 'Alice');
```

Use code with caution.

Behind the scenes:

Databricks writes a physical Parquet file on your cloud storage (S3/ADLS). Depending on your Databricks version, it may use a random physical string (like `col-a1b2c3d4...`) or the original name inside the Parquet file schema. Crucially, the data is tightly linked to **ID 1** and **ID 2**.

Step 3: You Rename the Column

You run the alter command:

sql

```
ALTER TABLE employee_table RENAME COLUMN first_name TO full_name;
```

Behind the scenes:

- **What Databricks does NOT do:** It does **not** open, read, or rewrite your giant Parquet data files.
- **What Databricks DOES do:** It only opens the tiny Delta Log file and updates the metadata map to say:
 - *“From now on, **ID 2** is called `full_name`.”*

This metadata change takes less than 10 milliseconds.

Step 4: You Query the Table

```
SELECT full_name FROM employee_table;
```

Behind the scenes:

1. Databricks looks at your query and sees you want `full_name`.
2. It checks the Delta Log map and asks: *“What ID is `full_name`?”* → The log answers: **ID 2**.
3. Databricks goes straight to the physical Parquet file, grabs the data belonging to **ID 2**, and hands it back to you.

Week 18:

Change Data Feed (CDF):

Delta logs → File level changes / Dataset level Changes

CDF is used to track row level changes between different versions.

automatically tracks row-level changes between table versions. Instead of performing expensive full-table scans to find out what changed, CDF allows downstream data pipelines to query and isolate exactly which rows were inserted, updated, or deleted

V0(version1 of table) to V2 (version2 of table)

We can able to view

Inserts

deletes

Pre-image and post-image for updates.

Process only changed data (incremental).

Ex:

```
%sql
DROP TABLE IF EXISTS demo;

CREATE TABLE demo
(
  id INT,
  name String,
  amount Double
);
```

```
%sql
INSERT INTO deltalake_demo.default.demo values
(1, 'Alice', 100.0),
(2, 'bob', 200.0),
(3, 'carol', 300.0);
```

```
%sql
DESCRIBE HISTORY deltalake_demo.default.demo;
```

Operation:

WRITE

CREATE TABLE

```
%sql
alter table deltalake_demo.default.demo set tblproperties(delta.enableChangeDataFeed
= true);
```

```
%sql
DESCRIBE HISTORY deltalake_demo.default.demo;
```

Operations:

SETTBLPROPERTIES

WRITE

CREATE TABLE

UPDATE:

```
%sql
UPDATE demo set amount=amount*1.1 where id=2;
```

```
%sql  
DESCRIBE HISTORY deltalake_demo.default.demo;
```

Operations:

OPTIMIZE

UPDATE

SETTBLPROPERTIES

WRITE

CREATE TABLE

Insert a record

```
%sql  
INSERT INTO demo VALUES (3, 'Adon', 400.0);
```

Delete a record

```
%sql  
DELETE FROM demo where id=1;
```

```
%sql  
DESCRIBE HISTORY deltalake_demo.default.demo;
```

operation

OPTIMIZE --7 (version)

DELETE

WRITE

OPTIMIZE

UPDATE

SET TBLPROPERTIES

WRITE

```
CREATE TABLE --0 (version)
```

Enable:

```
alter table deltalake_demo.default.demo set  
tblproperties(delta.enableChangeDataFeed = true);
```

Query:

##Now I want to see that after version 2 can you tell me what are updates inserts are happened.

```
%sql  
SELECT * FROM table_changes('demo',2);
```

_change_type	_commit_version	_commit_timestamp
update_preimage		
update_postimage		
insert		
delete		

```
%sql  
DELETE FROM DEMO WHERE ID=2;
```

```
%sql  
SELECT * FROM table_changes('demo',2);
```

_change_type

update_preimage

update_postimage

insert

delete

delete

I want to extract all the delete records

```
%sql
CREATE TABLE demo_deleted_data_batch
(
    id int,
    name string,
    amount double,
    _commit_type string,
    _commit_version long,
    _Commit_timestamp timestamp
)
using delta;
```

Now I want to insert from table_changes to demo_deleted_data_batch

```
%sql
INSERT INTO demo_deleted_data_batch
SELECT id,name,amount,_change_type,_commit_version,_commit_timestamp
from table_changes("demo",2)
where _change_type = 'delete';
```

Streaming

```

from pyspark.sql.functions import col
(
    spark.readStream.format("delta")
        .option("readChangeFeed", "true")
        .option("startingVersion", 2)
        .table("demo")
        .filter(col("_change_type") == "delete")
        .select("id", "name")
        .writeStream
        .outputMode("append")
        .option("checkpointLocation", "/Volumes/deltalake_demo/default/deltalake_demo/
checkpoint1/")
        .trigger(availableNow=True)
        .table("demo_deleted_data_streaming_auto")
)

```

Now If I delete another record in Source table:

```

%sql
DELETE FROM demo WHERE ID=3;

```

And run the insert with _change_type='delete' then

```

%sql
SELECT * FROM demo_deleted_data_batch;

```

id	name
3	carol
3	Adon
1	ALice
1	ALice
1	ALice
2	bob

2 bob

2 bob

We have duplicate data. After batch processing.

But when we do with streaming we have concept of CHECKPOINT.

```
from pyspark.sql.functions import col
(
    spark.readStream.format("delta")
        .option("readChangeFeed", "true")
        .option("startingVersion", 2)
        .table("demo")
        .filter(col("_change_type") == "delete")
        .select("id", "name")
        .writeStream
        .outputMode("append")
        .option("checkpointLocation", "/Volumes/deltalake_demo/default/deltalake_demo/
checkpoint1/")
        .trigger(availableNow=True)
        .table("demo_deleted_data_streaming_auto")
)
```

It will not process duplicate because of checkpoint it will only process the new changes to the source table.

If I want to automate my streaming job I want to use cluster not serverless.


```

from pyspark.sql.functions import col
(
    spark.readStream.format("delta")
        .option("readChangeFeed", "true")
        .option("startingVersion", 2)
        .table("demo")
        .filter(col("_change_type") == "delete")
        .select("id", "name")
        .writeStream
        .outputMode("append")
        .option("checkpointLocation", "/Volumes/deltalake_demo/default/deltalake_demo/
checkpoint1/")
        .trigger(processingTime="2 seconds")
        .table("demo_deleted_data_streaming_automatically")
)

```

Now I want to insert few more records:

```

%sql
INSERT INTO deltalake_demo.default.demo values
(4, 'ALice', 100.0),
(5, 'bob', 200.0),
(6, 'carol', 300.0),
(7, 'naresh', 200.0),
(8, 'Kumar', 230.00);

```

```

%sql
DELETE FROM DEMO where id=4;

```

```

%sql
DELETE FROM DEMO where name='naresh';

```

Now the streaming job with trigger(processingTime="2 second") will run automatically.
With 2 seconds gap.

availableNow=true

process all the data that is currently available.

runs as a finite job (like batch job then stops automatically)

if you re run it later with same checkpoint, it will pick up only new changes.

best usecase:

serverless (since continuous triggers are not supported)

periodic job (eg hourly or daily)-schedule notebook as job, no need to keep the cluster alive.

ProcessingTime="2 seconds"

From table creation onwards I want to track all the changes

```
%sql
CREATE TABLE demo_1
(
  id int,
  name string,
  amount double
)
using delta
tblproperties(delta.enableChangeDataFeed =true);
```

we can set the CFD property at spark session as well

```
spark.conf.set("spark.databricks.delta.properties.defaults.enableChangeDataFeed",
"true")
```

Checkpoint:

manage the state and processed offsets of a streaming query when consuming the **Delta Lake Change Data Feed (CDF)**. They guarantee **exactly-once processing** by saving progress, allowing the stream to fail and restart precisely where it left off.

CLONING:

cloning allows you to create a replica of an existing Delta Lake, Parquet, or Apache Iceberg table at a specific point in time. Unlike standard copy commands, cloning copies the exact metadata—including schema, partitioning, constraints, and properties—without needing to manually re-specify them.

Comparison of Clone Types

Feature	Deep Clone	Shallow Clone (Zero-Copy)
What is copied?	Both metadata and all underlying data files.	Only the metadata.
Data Storage	Duplicated completely into a new path.	References the original table's source files.
Cost & Speed	Slower and requires full storage costs.	Fast and incurs almost zero initial storage cost.
Independence	Fully independent.	Dependent on the source table's files remaining intact.
Primary Use Case	Production backups, data migration, disaster recovery.	Staging, testing, ML experimentation.

Clone can be of 2 types -:

- Shallow clone (only metadata is copied). **The clone table is pointing to source table only.**
- Deep clone (copies both metadata and data)

What Is Copied (The Metadata)

- **The Schema:** Column names, data types (e.g., Integer, String), and nullability.
- **The Partitioning:** How the data is organized structurally (e.g., partitioned by Year or Region).
- **Table Constraints:** Rules like NOT NULL or primary keys.
- **Table Properties:** Custom configurations and tags attached to the table.
- **File Pointers:** A list of paths pointing to the exact physical data files belonging to the original table at that specific second.

What Is NOT Copied (The Data)

- **The Parquet Files:** The actual storage blocks containing your millions of rows of raw data are completely untouched and uncopied.

Shallow COPY example:

```
%sql
DROP TABLE IF EXISTS orders_managed;

create or replace table order_managed
(
    order_id bigint,
    sku string,
    product_name string,
    product_category string,
    qty int,
    unit_price decimal(10,2)
);

alter table order_managed add constraint valid_qty check(qty>0);
```

Insert few records:

```
%sql
INSERT INTO order_managed (order_id, sku, product_name, product_category, qty,
unit_price) VALUES
(1001, 'ELEC-OFF-001', 'Ergonomic Wireless Mouse', 'Electronics', 2, 29.99);
```

Insert few more:

```
%sql
INSERT INTO order_managed (order_id, sku, product_name, product_category, qty,
unit_price) VALUES
(1011, 'ELEC-CHG-015', 'Fast Charging Wall Adapter', 'Electronics', 2, 18.99);
```

```
%sql
delete from order_managed where order_id='1004';
```

```
%sql
DESCRIBE HISTORY order_managed;
```

```
%sql
update order_managed
set qty=3,
unit_price=3299.00
where order_id='1007'
```

one deletion vector and one parquet will be created.. one optimize would be created. – For UPDATE command.

```
%sql
INSERT INTO order_managed (order_id, sku, product_name, product_category, qty,
unit_price) VALUES
(1014, 'AUTO-PHN-067', 'Magnetic Car Phone Mount', 'Automotive', 1, 12.95),
(1015, 'TOY-GME-551', 'Strategy Board Game', 'Toys & Games', 1, 39.99);
```

```
%sql
describe detail order_managed;
-- here we can see what are files of data we fetching. ye ye files ni use chetsamu
when we hit "select * from"..
-- check numFiles parameter
```

```
%sql
select distinct _metadata.file_path from order_managed;
```

I want to Do a shallow clone:

```
%sql
DROP TABLE IF EXISTS orders_managed_sclone;
create or replace table orders_managed_sclone SHALLOW CLONE order_managed;
```

```
%sql
DESCRIBE DETAIL orders_managed_sclone;

-- Once I checked in azure storage it will copy only metadata not parquet files.check
it properly
```

```
%sql
DESCRIBE history orders_managed_sclone;

-- operationParameters  -- here we can check at what version we clone from the source
table
-----
-- object
-- source: "deltalake_demo.default.order_managed"
-- sourceVersion: "16"
-- isShallow: "true"
```

If I check order_managed and orders_managed_sclone both tables are pointing to the same files. it means the clone table copy only metadata not actually data.

If i insert few records in source table

```
%sql
INSERT INTO order_managed (order_id, sku, product_name, product_category, qty,
unit_price) VALUES
(1089, 'ELEC-OFF-001', 'Ergonomic Wireless Mouse', 'Electronics', 2, 29.99),
(1099, 'HOME-KIT-042', 'Stainless Steel Water Bottle', 'Home & Kitchen', 1, 15.50);
```

```
%sql
DELETE FROM orders_managed_sclone WHERE ORDER_ID='1007';
```

```
%sql  
describe history orders_managed_sclone;
```

operation

DELETE

CLONE

```
%sql  
select * from orders_managed_sclone;  
-- here we don't have record with 1007 bcz it is deleted.
```

```
%sql  
SELECT * from order_managed;  
-- here we can able to view the record with 1007
```

How the Delete Works (Logical, Not Physical)

Delta Lake uses ACID transactions and logical deletes. When you ran DELETE FROM clone WHERE id=1007:

- Databricks did not go into the source file and erase the data.
- Instead, it wrote a new entry only in the clone table's transaction log stating: "The file containing record 1007 is now marked as removed/rewritten for this table."
- "Because this change was only written to the clone's log, the source table's log remains completely untouched."

3. Reading the Tables

When you query the clone:

The clone's transaction log tells the system to ignore or hide record 1007. Therefore, you do not see it.

When you query the source: The source table reads its own original transaction log, which has no record of your deletion. Therefore, record 1007 is still visible

What if I delete all the records from the source table order_managed and then do a VACUUM RETAIN 0 hours;

```
%sql  
DELETE from order_managed;
```

```
%sql  
VACUUM order_managed DRY RUN;
```

```
%sql  
VACUUM order_managed RETAIN 0 hours dry run;
```

*The specified VACUUM retention period is too low and may corrupt this Delta table if any writes are in progress. If no operations (insert, upsert, delete, optimize) are running, you can disable this safety check by setting: delta.retentionDurationCheck.enabled = false
Otherwise, use a retention period of at least 168 hours. SQLSTATE: 22003*

```
%sql  
ALTER TABLE order_managed SET TBLPROPERTIES(  
    'delta.deletedFileRetentionDuration'='interval 0 hours'  
)
```



```
%sql
VACUUM order_managed DRY RUN;
-- remove only files which are not referred.and it will not deleted the clone
referred files as well.
```

```
%sql
VACUUM order_managed;
```

Clone table refer chese files kuda delete kavu. Shallow clone table should not be effected.

```
%sql
SELECT * FROM orders_managed;

-- It is shows 0 records.
```

```
%sql
SELECT * FROM orders_managed_sclone;

-- It is still show records. beacuse the refereed files are not deleted.
```

```
%sql
SELECT * FROM order_managed@v8;
```

Above query will results yala ante at version 8 we created clone. So I clone ni refer chese files inka unnaye kada anduke manaki result vastundhi.

In case eppdu clone chesina veersion kakakunda vere version like v9 or v10 thesukunte manaki output em radhu.

```
%sql
SELECT * FROM order_managed@v5;
```

**Cannot time travel beyond delta.deletedFileRetentionDuration (0 HOURS) set on the table.
SQLSTATE: 0AKDC**

```
%sql
ALTER TABLE order_managed_sclone SET TBLPROPERTIES(
    'delta.deletedFileRetentionDuration'='interval 0 hours'
)
```

```
%sql
VACUUM order_managed_sclone DRY RUN;
-- remove only files which are not referred.and it will not deleted the clone
referred files as well.
```

```
%sql
VACUUM order_managed_sclone;
```

It will delete all the files. Enduku ante source table already delete chesamu. No refer to source table. So anni files delete ayipthoye.

If clone table refereeing something source table cannot remove

If source table refereeing something clone table cannot remove.

If not refereeing any files in in any table either source table or clone table then only vaccum command will remove.

DROP

What if I drop source table:

```
%sql
drop table order_managed;
-- it should not impact the clone table bcz we undrop till 7 days that's the reason
we can able to view clone table data as well.
```

```
%sql
select * from orders_managed_sclone;

# it will works only 7 days due to retention policy.
```

What happens when the source data changes after shallow clone ?

The source and clone have independent Delta transaction logs. Insert, update and delete operations on one table do not automatically affect the other.

However, a shallow clone share the underlying data files with source . SO physical file lifecycle operations such VACUUM can affect the clone if files it references are removed.

Q/A

If I insert data into source after creating a shallow clone, will clone see it.

No

A shallow clone gets a snapshot of source at time of cloning.

10:00 – clone created.

10:05 → Insert into source.



Source – new record.

Clone → No new record.

If I drop shallow clone what happens to the source data ?

The source remains unaffected.

Dropping the clone removes the clone's table metadata/transaction log. It does not delete the source table.

If I drop source table, can the shallow clone still work ?

It may continue to work as long as the underlying data files it references still exist.

But the clone is **not independent** because it shares the source's physical data file.

If those files are later removed through cleanup/VACUUM the clone can lose access to them.

Why can VACUUM on the source potentially break a shallow clone ?

Because VACUUM removes old, unreferenced data files according to the retention rules.

The source may consider a file unnecessary, while the shallow clone may still need that same:

Source Says:

"I don't need FILE_A anymore"

Clone Says:

"I still need FILE_A".

If the source table is VACUUMed what risk does the shallow clone have ?

Source



VACUUM



Old file deleted



Clone still references old file.



Potential clone failure.

DEEP CLONE:

Deep Clone = new Delta table + new Delta log + independent physical data files.

```
%sql
describe detail order_managed;
```

```
%sql
SELECT DISTINCT _metadata.file_path FROM order_managed;
```

```
%sql
DROP TABLE IF EXISTS order_managed_dclone;

CREATE OR REPLACE TABLE order_managed_dclone DEEP CLONE order_managed;

DESCRIBE DETAIL order_managed_dclone;
```

it copy only the the files which we are referred.

```
%sql
SELECT DISTINCT _metadata.file_path FROM order_managed_dclone;
```

check above source table and dclone table have same files which are referring.

What happens when you insert records?

###

Scenario A: You insert records into the Source Table New Parquet files are written only to the Source Table's storage path. The Clone Table remains unchanged. It will not see the new records.

###

Scenario B: You insert records into the Clone Table New Parquet files are written only to the Clone Table's storage path. The Source Table remains unchanged. It will not see the new records

- any changes made to either deep clone or shallow clone , affects only the clone themselves and not the source table.
- cloning a table is not the same as CTAS create table as select *

create new_table as

select * from orders_managed; (it is not equal to deep clone)

- you can use deep clone to preserve the state of a table at a certain point in time for archival purpose.

create or replace table archive_table DEEP CLONE my_prod_table;

- to test a workflow on a prod table without corrupting the table, you can easily create a shallow clone.

this allows you to run arbitrary workflows on the cloned table that contains all the prod data but does not affect production workloads.

Ex: The Scenario: Testing a GDPR Deletion Script

you have a massive production table called prod.customers containing 10 TB of data. You need to test a new automated script that deletes deactivated user accounts to comply with privacy laws. If your script has a bug (like a missing WHERE clause), it could accidentally wipe out your entire production database. Instead of risking production or waiting hours to copy 10 TB for a test environment, you use a Shallow Clone.

- shallow clones on external tables must be external tables. shallow clone on managed table must be managed table.

- for managed tables, dropping the source table breaks the target table for shallow clones. (remember retention period)

- shallow clones on external table are not impacted by dropping the source.::: if your source table is an External Table, dropping it will delete only the table registration name from the metastore catalog, leaving the actual data files safe and untouched in cloud storage

```
%sql
-- Standard deep clone
CREATE TABLE target_catalog.target_schema.target_table
DEEP CLONE source_catalog.source_schema.source_table;

-- Replace an existing target table incrementally
CREATE OR REPLACE TABLE target_schema.target_table
DEEP CLONE source_schema.source_table;

-- Clone from a specific historical version (Time Travel)
CREATE TABLE target_schema.archived_table
DEEP CLONE source_schema.source_table VERSION AS OF 12;
```

```
from delta.tables import DeltaTable

# Perform the deep clone using DeltaTable API
deltaTable = DeltaTable.forName(spark, "source_schema.source_table")
deltaTable.clone("target_schema.target_table", isShallow=False)
```

UNIFORM:

delta is very famous lakehouse file format.

Iceberg

the only difference between delta and iceberg is about the metadata.

Core Concept

UniForm (Universal Format) is a metadata translation technology developed by Databricks. It allows a single dataset stored in the Delta Lake format to be read natively by engines that only support Apache Iceberg or Apache Hudi.

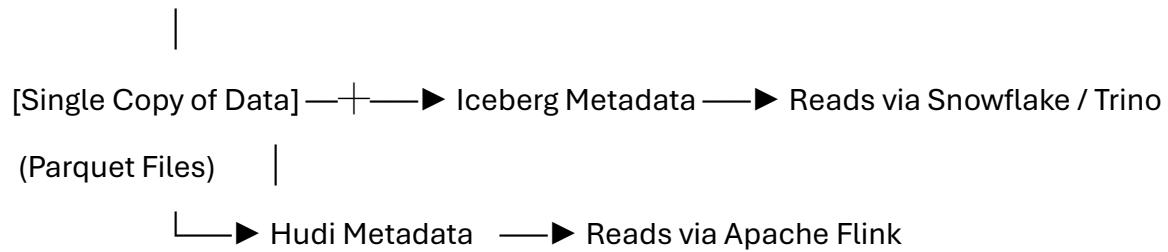
The Problem It Solves

Historically, the big three Lakehouse formats—Delta Lake, Apache Iceberg, and Apache Hudi—created architectural silos. Even though all three formats store their raw data in identical standard Apache Parquet files, they cannot talk to one another because they use completely different metadata transaction logs to track which Parquet files belong to a table.

Before UniForm, if you had a Databricks platform (Delta) and a Snowflake platform (Iceberg), you had to duplicate your data pipelines and pay twice for cloud storage to serve both engines.

- > You write data normally to Databricks using Delta Lake.
- > As Databricks writes the Delta metadata, it automatically and asynchronously generates the equivalent Iceberg/Hudi metadata files pointing to those exact same Parquet data files.
- > Zero data is duplicated. You only pay for a single set of Parquet storage files

└─▶ Delta Metadata ──▶ Reads via Databricks / Spark



this functionality was previously called delta lake universal format (uniform)

Multiple formats, single copy of data.

Small File problem:

you have 10000 files each file is of roughly 1mb

```
select avg(sales) from order_managed;
```

Before your engine can calculate the average, the Spark driver must list, open, read, and close every single file. it is overhead.

100 files of each file have 100 mb each

you have a table

and every one hour ther is some incrementa updatel which is coming...

```
create or replace table order_managed
(
  order_id bigint,
  sku string,
  product_name string,
  product_category string,
  qty int,
  unit_price decimal(10,2)
)
USING DELTA
TBLPROPERTIES (
  delta.autoOptimize.optimizeWrite =false,
  delta.autoOptimize.autoCompact=false
)
```

```
query="select avg(unit_price) as avg_price from order_managed"
res=spark.sql(query).collect()
print(res)
```

```
# in performance the FILES AND partitions :
# files read 200
# and there is no files pruned because it is open, read and close each and every
file.
```

```
%sql
OPTIMIZE order_managed;

-- take screenshot below for reference check numFilesAdded:1 and numFilesRemoved: 200
```

Code editor showing SQL execution results for query 15:

```
%sql
OPTIMIZE order_managed;
--take screenshot below for reference check numFilesAdded:1 and numFilesRemoved: 200
```

See performance (1) | Checked for improvements

Table	path	metrics
1		object numFilesAdded: 1 numFilesRemoved: 200 > filesAdded: {"min": 54815, "max": 54815, "avg": 54815, "totalFiles": 1, "totalSize": 54815} > filesRemoved: {"min": 2542, "max": 2628, "avg": 2575.43, "totalFiles": 200, "totalSize": 515086}

Code editor showing SQL execution results for query 16:

```
%sql
DESCRIBE HISTORY order_managed;
```

See performance (1) | Checked for improvements

version	timestamp	userId	userName	operation	operationPa
2	2026-08-03T04:39:07.000+00:...	11135336969592...	nareshboda112@gmail.co...	OPTIMIZE	> {"predicate":
1	2026-08-03T02:10:22.000+00:...	11135336969592...	nareshboda112@gmail.co...	WRITE	> {"mode": "App
0	2026-08-03T02:07:00.000+00:...	11135336969592...	nareshboda112@gmail.co...	CREATE OR REPLACE TABLE	> {"partitionBy"

3 rows | 2.559s runtime | Refreshed 7d ago

This result is stored as `_sqldf` and can be used in other Python and SQL cells. [Send feedback](#)

```
query="select avg(unit_price) as avg_price from order_managed"
res=spark.sql(query).collect()
print(res)
# check performance tab the numoif files read is : 1
```

the file ideal size is: 16mb to 1gb (ideal file size)

less than 16 is small file

greater than 1gb is big file

spark.databricks.delta.optimize.maxFileSize ~ 1GB

set spark.databricks.delta.optimize.maxFileSize=1GB

bit packing algo:

100mb

300mb

300mb

100mb

600mb

300

lets say if we optimize 1GB:

internally it sorts in desc order

600mb

300mb

300mb

300mb

100mb

100mb

bin 1 --> 600 mb,300mb,100mb

bin2 --> 300mb, 300mb, 100 mb

AUTO OPTIMIZE

```
%sql
DROP TABLE IF EXISTS order_managed;

create or replace table order_managed
(
  order_id bigint,
  sku string,
  product_name string,
  product_category string,
  qty int,
  unit_price decimal(10,2)
)
USING DELTA;
```

```
from pyspark.sql import Row
import random

# 1. Generate 10,000 synthetic records in memory
data = []
for i in range(10000):
    data.append(Row(
        order_id=int(2000 + i),
        sku=f"SKU-{random.randint(100, 999)}",
        product_name=f"Product-{random.randint(1, 10)}",
        product_category="Electronics" if i % 2 == 0 else "Appliances",
        qty=int(random.randint(1, 10)),
        unit_price=float(round(random.uniform(5.0, 100.0), 2))
    ))

# 2. Convert to DataFrame
df = spark.createDataFrame(data)

# 3. Force 200 partitions to guarantee 200 small physical files
df.repartition(200).write.format("delta").mode("append").saveAsTable("order_managed")
```

```
print("Successfully inserted 10,000 records split across 200 tiny files.")
```

```
query="select avg(unit_price) as avg_price from order_managed"  
res=spark.sql(query).collect()  
print(res)  
# check performance tab the numoif files read is :
```

Diff between Optimize and Auto optimize:

Manual OPTIMIZE : you tell Databricks when to optimize.

Auto Optimize = Databricks automatically handles file optimization during writes/ background maintenance.

Manual Optimize : **OPTIMIZE my_table;**

Auto optimize:

Auto optimization is designed to reduce the need for you to manually manage small files.

In modern Databricks, this is mainly associated **with optimized writes and automatic compaction.**

Auto Optimize completely replaces OPTIMIZE:

Auto Optimize reduces the need for manual file management by optimizing writes and performing automatic compaction, while manual OPTIMIZE gives us explicit control to compact an existing table and perform targeted optimizations such as Z-ordering.

OPTIMIZE: combines small files into larger files.

Z-ORDER : rearrange the data inside those files to make queries on specific columns faster.

PARTITIONING:

Data skipping

1000 files

800 files

20 files

Data layout of your table:

1.partitioning

2.Zorder

sales_table:

select _____ from sales where country= ?

50 such folders

country=India

country=US

country=Australia

Hive style partitioning:

```
%sql
DROP TABLE IF EXISTS orders_managed_partitioned;

create or replace table order_managed_partitioned
(
    order_id bigint,
    sku string,
    product_name string,
    product_category string,
    qty int,
    unit_price decimal(10,2),
    country string
)
USING DELTA
PARTITIONED BY (country);
```

```
%sql
insert into order_managed_partitioned
select * from order_managed;
```

```
%sql
describe detail order_managed_partitioned;

-- we can check the column "partitionColumns"
```

it is shows all the partition columns:

```
%sql
show partitions order_managed_partitioned;
```

```
%sql
select product_category, sum(qty) as total_qty
from order_managed_partitioned
where country='India'
group by product_category;
```

```
%sql
select count(*) from order_managed_partitioned;
-- it checks the only metadata not even read the files.
```



```
%sql
select sum(qty) as total_qty_over_100
from order_managed_partitioned
where unit_price>100;
-- it can read each and every file in order to get the sum value.
```

```
%sql
select avg(qty) from order_managed_partitioned;

-- Files & partitions
-- Files read    5
-- Files pruned  0
-- Partitions read 5

-- 0 files pruned so it will read each and every file to get avg value.
```

```
df.write.format("delta") \
  .mode("overwrite") \
  .option("overwriteSchema", "true") \
  .partitionBy("new_partition_column") \
  .saveAsTable("my_catalog.my_schema.my_table")
```

Challenges:

- you can apply on columns with low cardinality (low cardinality)
- no obvious benefits for data skipping.
- if usage pattern changes partitioned needs to be changed.
- to change partitioning, table must be rewritten.
- can have small file problem.
- high chances of data skewness.

Rigid Architecture: Partition columns cannot be changed or altered after table creation without completely rewriting the entire dataset.

1-TB Requirement: Databricks provides zero performance benefits—and often slows down operations—for tables containing less than 1 Terabyte of data.

File System Bottlenecks: Creating too many physical nested folders causes cloud object storage (S3/ADLS) to slow down drastically during directory listing phases.

The Small File Problem: Partitioning by high-cardinality columns (many unique values) creates thousands of tiny files, which can exhaust the Spark driver's memory.

Under-Partitioning: Choosing a column with too few unique values results in massive individual folders, making partition filtering completely useless.

Example:

How It Fails (The Mechanics)

The Goal of Partitioning: Partitioning is designed to let Databricks use Partition Pruning. This means if you query for one specific category, the engine instantly ignores 90% of the folders on disk and only reads the 10% it needs.

The Under-Partitioned Reality: If you partition a 100-million-row table by a column like `Is_Active` (which only has two values: True or False), you only create two folders

The Scan Bottleneck: When you write `WHERE Is_Active = True`, Databricks successfully skips the False folder, but it still has to scan a single, massive folder containing 90 million rows

Strict Query Dependencies: Performance gains only happen if users explicitly include the exact partition columns inside their SQL WHERE clauses.

Unmanageable Data Skew: Traditional partitioning cannot balance data; if one folder contains 90% of your records, a single cluster node does all the work while the rest sit idle.

Z ordering:

Z-Ordering is a multidimensional data clustering technique used to physically reorganize data within Delta Lake files to dramatically speed up query performance. It works by mapping multidimensional data into a single dimension while preserving data locality, ensuring that related information is co-located in the same set of files.

lets say the filter column is "trip distance"

file <min distance> <max_distance>

file1 -> 0,500

file2 -> 2,400

file3-> 10,450

file4 -> 5,350

file5-> 20,100

trip_distance > 50 and trip_distance < 100

after applying Z order

file1 -> 0,30

file2-> 31,80

file3 -> 81,200

file4 -> 201,400

file5-> 401, 500

```
%sql
CREATE TABLE IF NOT EXISTS nyc_taxi_trips (
  vendor_id STRING,
  pickup_datetime TIMESTAMP,
  dropoff_datetime TIMESTAMP,
  passenger_count INT,
  trip_distance DOUBLE,
  pickup_longitude DOUBLE,
  pickup_latitude DOUBLE,
  rate_code_id INT,
  store_and_fwd_flag STRING,
  dropoff_longitude DOUBLE,
  dropoff_latitude DOUBLE,
  payment_type STRING,
  fare_amount DOUBLE,
  extra DOUBLE,
  mta_tax DOUBLE,
  tip_amount DOUBLE,
  tolls_amount DOUBLE,
  total_amount DOUBLE
)
USING DELTA
TBLPROPERTIES (
  delta.autoOptimize.optimizeWrite =false,
  delta.autoOptimize.autoCompact=false
);
```

below we are inserting sample databricks data files and insert into above table.

```

table_location='dbfs:/databricks-datasets/nyctaxi/tables/nyctaxi_yellow'
all_files = [f.path for f in dbutils.fs.ls(table_location) if
f.path.endswith(".parquet")]
files20=all_files[:20]
print("using these 10 files:",files20)

volume_path='dbfs:/Volumes/deltalake_demo/default/deltalake_demo/z_order_demo/'
df_20=spark.read.parquet(*files20)
df_20.repartition(200).write.format("delta").mode("overwrite").save(volume_path)

```

```

%sql
INSERT INTO nyc_taxi_trips
SELECT * FROM delta.`/Volumes/deltalake_demo/default/deltalake_demo/z_order_demo/`;

```

```

%sql
select count(*) from nyc_taxi_trips where trip_distance > 100;

-- Files & partitions
-- Files read    50
-- Files pruned  0
-- Partitions read 0

```

```

%sql
select min(trip_distance),max(trip_distance),_metadata.file_name
from nyc_taxi_trips
group by _metadata.file_name
order by min(trip_distance);

```

4 days ago (5s) 31 SQL

```
%sql
select min(trip_distance),max(trip_distance),metadata.file_name
from nyc_taxi_trips
group by metadata.file_name
order by min(trip_distance);
```

See performance (1) ✓ Checked for improvements

	1.2 min(trip_distance)	1.2 max(trip_distance)	file_name
1	0	3008331.8	part-00008-6f752f64-b219-466b-8e69-62ca4455a0c5.c000.zstd.parquet
2	0	0.27	part-00000-c1d00931-16c9-4300-952e-32405d008133.c000.zstd.parquet
3	0	7000000	part-00027-17bc6ce2-be0b-4c65-b58a-19ac17080d1e.c000.zstd.parquet
4	0	3008.3	part-00007-01f24d1b-d944-4aa3-a073-af9af9dbaa9f.c000.zstd.parquet
5	0	6746333.1	part-00022-bb3bfaab-315e-4f29-b4e7-c7bb4ee61976.c000.zstd.parquet
6	0	8331800	part-00006-24206890-699e-4dbf-b5da-181fbb4e95bc.c000.zstd.parquet
7	0	1800503.3	part-00018-9abd58ae-91e3-4395-ab41-e27742c1440d.c000.zstd.parquet
8	0	3008.3	part-00019-3624368a-8376-447c-a00e-102641077d0b.c000.zstd.parquet
9	0	9025000	part-00044-571f9352-2587-41f7-a35f-43f1b9b68e18.c000.zstd.parquet
10	0	10000	part-00004-78f6706a-89b9-4247-9984-3aeb790e30b3.c000.zstd.parquet
11	0	8335700	part-00021-d0d94be2-8205-4cad-a7df-bfc2d4efc50f.c000.zstd.parquet
12	0	302	part-00043-006c7870-53e8-42d0-9603-f301d3f28caa.c000.zstd.parquet
13	0	493164.3	part-00049-9a573302-04c3-48ae-adfa-93e7d8d77627.c000.zstd.parquet

If we check the min and max values are messy. Check once we run z-order command.

```
%sql
optimize nyc_taxi_trips zorder by (trip_distance);

-- trip_distance is high cardinality column

-- numFilesAdded: 62
-- numFilesRemoved: 50
```

For the volume part we can do like below way:

```
%sql
optimize delta.`/Volumes/deltalake_demo/default/deltalake_demo/z_order_demo/` zorder
by (trip_distance);

-- numFilesAdded: 20
-- numFilesRemoved: 200
```

```
%sql
select count(*) from
delta.`/Volumes/deltalake_demo/default/deltalake_demo/z_order_demo/` where
trip_distance > 100;

-- Files & partitions
-- Files read    1
-- Files pruned 19
```

```
%sql
select count(*)
from delta.`/Volumes/deltalake_demo/default/deltalake_demo/z_order_demo/`
where trip_distance >5 and trip_distance<7;

-- Files & partitions
-- Files read    2
-- Files pruned 18
-- Partitions read 0
```

Z ordering is a technique to colocate related information in the same set of files.

this co-locality is automatically used by delta lake on databricks for data skipping.

if you expect a column to be commonly used that has high cardinality then use z order by.

partitioning and z order serve same purpose.to skip some data.

partitioning (low cardinality columns)

zordering (on high cardinality columns)

Liquid Clustersing

Partitioning

Zordering

We have to struggle to determine the partitioning column and zorder column.

If the reading pattern changes then we have to rewrite all the data.

To resolve above issues we are using liquid clustering....

2020 – small file

2021 -- small file

2022 -- large file

2023 -- medium file

2024 -- small file

2025 -- small file

Medium size is the ideal size

2020,2021 – make as ideal size

2022-1, 2022-2

2023

2024,2025 - make as ideal size (medium)

	01-01	02-01	03-01
India	small	small	medium
US	small	very small	very small
Australia	small	very small	very small

Liquid clustering can be a good replacement for partitioning and zorder.

```
%sql
use catalog my_catalog;
```

```
%sql
CREATE TABLE IF NOT EXISTS nyc_taxi_trips (
  vendor_id STRING,
  pickup_datetime TIMESTAMP,
  dropoff_datetime TIMESTAMP,
  passenger_count INT,
  trip_distance DOUBLE,
  pickup_longitude DOUBLE,
  pickup_latitude DOUBLE,
  rate_code_id INT,
  store_and_fwd_flag STRING,
  dropoff_longitude DOUBLE,
  dropoff_latitude DOUBLE,
  payment_type STRING,
  fare_amount DOUBLE,
  extra DOUBLE,
  mta_tax DOUBLE,
  tip_amount DOUBLE,
  tolls_amount DOUBLE,
  total_amount DOUBLE
)
USING DELTA
TBLPROPERTIES (
  delta.autoOptimize.optimizeWrite =false,
  delta.autoOptimize.autoCompact=false
);
```

```
%sql
create volume new_liquid_clustering;
```

```
table_location='dbfs:/databricks-datasets/nyctaxi/tables/nyctaxi_yellow'
all_files = [f.path for f in dbutils.fs.ls(table_location) if
f.path.endswith(".parquet")]
files20=all_files[:20]
```

```
print("using these 10 files:",files20)

volume_path='/Volumes/my_catalog/default/new_liquid_clustering/demo/'
df_20=spark.read.parquet(*files20)
df_20.repartition(200).write.format("delta").mode("overwrite").save(volume_path)
```

Either we can write to volume path or saveAsTable("<table_name")

```
%sql
INSERT INTO nyc_taxi_trips
SELECT * FROM delta.`/Volumes/my_catalog/default/new_liquid_clustering/demo/`;
```

```
%sql
select * from nyc_taxi_trips where trip_distance >100;

--Files & partitions
-- Files read    50
-- Files pruned  0
-- Partitions read 0
```

```
%sql
alter table nyc_taxi_trips cluster by (trip_distance);
```

```
%sql
describe history nyc_taxi_trips;
```

```
%sql
select * from nyc_taxi_trips where trip_distance >100;
-- Files & partitions
-- Files read    50
-- Files pruned  0
-- Partitions read 0
--
-- manam liquid clustering enable chesina manaki adhe effect chupichadam ledhu why
because we don't activated that.. to actiavte we need to use optimize command.
```

```
%sql
optimize nyc_taxi_trips;

-- numFilesAdded: 81
-- numFilesRemoved: 50
```

```
%sql
select * from nyc_taxi_trips where trip_distance >100;

-- Files & partitions
-- Files read 1
-- Files pruned 80
-- Partitions read 0
```

```
%sql
select min(trip_distance),max(trip_distance),_metadata.file_name
from nyc_taxi_trips
group by _metadata.file_name
order by min(trip_distance);

-- ekkada manam min and max trip_distance values proper ga laydown ayyaye... check it
bro,,,
```

```
%sql
describe detail nyc_taxi_trips;
--clusteringcolumns: ["trip_distance"]
```

```
%sql
alter table nyc_taxi_trips CLUSTER BY AUTO;
```

In Databricks, **CLUSTER BY AUTO** enables [Automatic Liquid Clustering](#).

Instead of a human developer having to manually pick columns for data layout optimization (which used to be done via partitioning or ZORDER), you hand over that responsibility to Databricks.

```
%sql
alter table nyc_taxi_trips CLUSTER BY None;
```

In the context of Databricks and Liquid Clustering, **NONE** is used to completely **turn off and remove clustering for a table**.

Liquid Clustering DEMO 2:

```
%sql
CREATE TABLE IF NOT EXISTS nyc_taxi_trips_01 (
  vendor_id STRING,
  pickup_datetime TIMESTAMP,
  dropoff_datetime TIMESTAMP,
  passenger_count INT,
  trip_distance DOUBLE,
  pickup_longitude DOUBLE,
  pickup_latitude DOUBLE,
  rate_code_id INT,
  store_and_fwd_flag STRING,
  dropoff_longitude DOUBLE,
  dropoff_latitude DOUBLE,
  payment_type STRING,
  fare_amount DOUBLE,
  extra DOUBLE,
  mta_tax DOUBLE,
  tip_amount DOUBLE,
  tolls_amount DOUBLE,
  total_amount DOUBLE
)
USING DELTA
PARTITIONED BY (vendor_id)
TBLPROPERTIES (
  delta.autoOptimize.optimizeWrite =false,
  delta.autoOptimize.autoCompact=false
);
```

```
table_location='dbfs://databricks-datasets/nyctaxi/tables/nyctaxi_yellow'
all_files = [f.path for f in dbutils.fs.ls(table_location) if
f.path.endswith(".parquet")]
```

```

files10=all_files[:10]
print("using these 10 files:",files10)

volume_path='/Volumes/my_catalog/default/new_liquid_clustering/new_demo/'
df_20=spark.read.parquet(*files20)
df_20.repartition(200).write.format("delta").mode("overwrite").saveAsTable("nyc_taxi_
trips_01")

```

```

%sql
INSERT INTO nyc_taxi_trips_01
SELECT * FROM delta.`/Volumes/my_catalog/default/new_liquid_clustering/new_demo/`;

```

```

%sql
select count(*) from nyc_taxi_trips_01 where trip_distance > 100;

-- Files & partitions
-- Files read    33
-- Files pruned  55
-- Partitions read  2

-- with partition we are reading 2 partitions

```

Lets try with Zorder

```

%sql
optimize nyc_taxi_trips_01 zorder by (trip_distance);

-- numFilesAdded: 29
-- numFilesRemoved: 87

```

```
%sql
select count(*) from nyc_taxi_trips_01 where trip_distance > 100;

-- after applying z order as well below are the stats

-- Files & partitions
-- Files read    2
-- Files pruned 28
-- Partitions read  2

-- the partitions read still is 2

-- here we did the partition by vendor_id column and zorder by trip_distance column
```

lets drop the table and re create with liquid clustering option

```
%sql
drop table nyc_taxi_trips_01;
```

```
%sql
CREATE TABLE IF NOT EXISTS nyc_taxi_trips_01 (
  vendor_id STRING,
  pickup_datetime TIMESTAMP,
  dropoff_datetime TIMESTAMP,
  passenger_count INT,
  trip_distance DOUBLE,
  pickup_longitude DOUBLE,
  pickup_latitude DOUBLE,
  rate_code_id INT,
  store_and_fwd_flag STRING,
  dropoff_longitude DOUBLE,
  dropoff_latitude DOUBLE,
  payment_type STRING,
  fare_amount DOUBLE,
  extra DOUBLE,
  mta_tax DOUBLE,
  tip_amount DOUBLE,
  tolls_amount DOUBLE,
  total_amount DOUBLE
)
USING DELTA
```

```
CLUSTER BY (vendor_id,trip_distance)
TBLPROPERTIES (
    delta.autoOptimize.optimizeWrite =false,
    delta.autoOptimize.autoCompact=false
);
```

```
%sql
INSERT INTO nyc_taxi_trips_01
SELECT * FROM delta.`/Volumes/my_catalog/default/new_liquid_clustering/new_demo/`;
```

```
%sql
describe detail nyc_taxi_trips_01;

-- clusteringColumns
-- 0: "vendor_id"
-- 1: "trip_distance"
```

```
%sql
select count(*) from nyc_taxi_trips_01 where trip_distance > 100;

-- Files & partitions
-- Files read    2
-- Files pruned 51
-- Partitions read 0

-- after applying the liquid clustering
```

%md

benefits of liquid clustering

1. incremental clustering.. if our data grows clustering automatically applies
2. flexibility to change the columns (auto)
3. avoid rewriting all the data
4. avoid write conflicts

lets say partitioning

5 partitions

India -- we are performing merge operation -- Developer1

India -- another developer 2 performing merge operation

so one developer query will succeed and another developer query will fail.

the following are the scenarios when you can benefit from liquid clustering:

- tables after filtering by high cardinality columns
- tables with significant skew in data distribution
- tables with access patterns that change over time (using AUTO keyword)
- tables with concurrent write requirements.
- tables that require a lot of maintainance and tuning efforts.

you can enable liquid clustering on a brand new table

or on a existing table

after liquid clustering is enables, run optimize jobs as usual to incrementally cluster new data.

if you have predictive optimization enabled. optimize runs automatically.

it works in a ideal data layout : that means

files between 16mb to 1GB

non overlapping data ranges

cost of maintaing the data layout must be cheap

above 3 things will be fullfill to use liquid clustering...

what is predictive optimization:

=====

it removes the need to manually manage the maintainance operations for UC (unity catalog) managed tables on azure databricks.

1. optimize -- triggers incremental clustering for liquid enabled tables.
2. it improves query performance by optimizing file sizes.

Vacuum --> reduces storage costs by deleting data files no longer referenced by the table.

Analyse : triggers incremental update iif stats to improve query performance.

the above 3 commands will run as part of predicative optimization.

it uses serverless compute to perform above background operations. your account is billed for compute associated with these workloads using a serveless jobs sku.

WEEK 19 : Lakeflow and Autoloader:

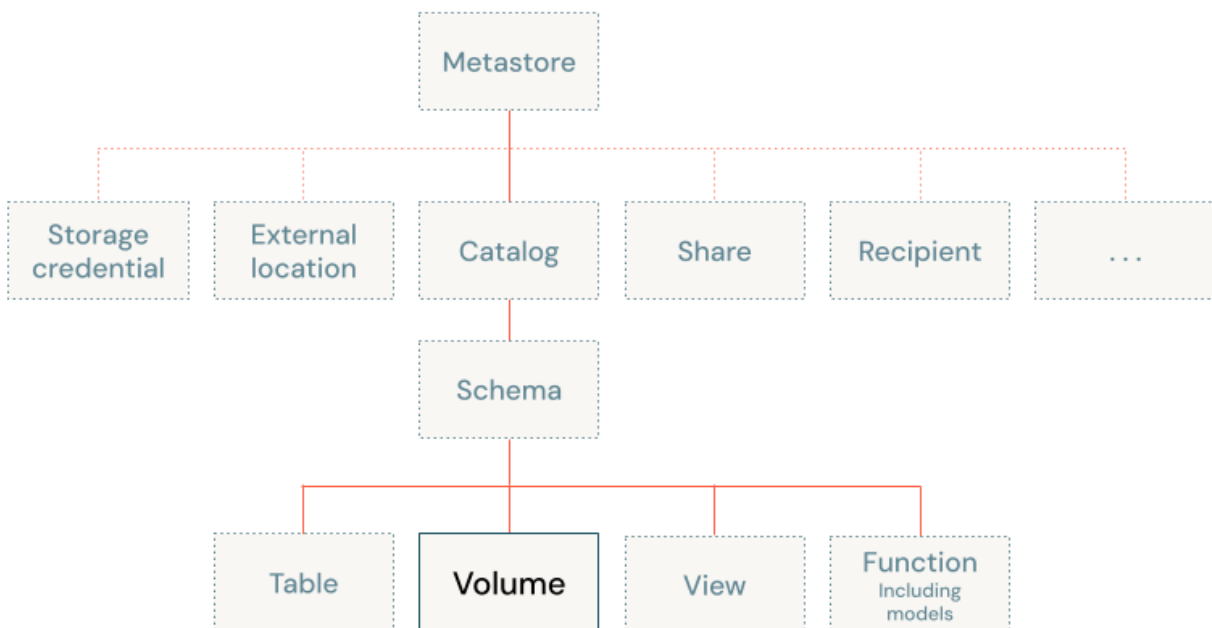
Volumes are unity catalog objects that enables governance over non-tabular datasets.

While tables govern tabular data, volumes govern non-tabular data of any format , including structured, Semi structured or unstructured.

Databricks recommends using volumes to govern all non-tabular data.

2 types of volumes:

- Managed
- External



Lakeflow Connect: Data Ingestion Overview:

Lakeflow – we have lake flow connect/ Declarative Pipelines / Jobs

Data Processing engine (Apache Spark)

Unified Governance (unity catalog)

Optimized Storage(Delta lake)

If we something bring to Delta Lake we can do:

- ACID properties
- DML
- Time Travel
- Schema Evolution and Enforcement.

Lakeflow – we have lake flow connect/ Declarative Pipelines / Jobs

Lakeflow provides a unified platform for

- Data ingestion (Using lakeflow connect)
- Transformation (using Declarative pipeline) earlier called as DLT
- Orchestration (using Lake flow jobs) earlier called as workflows.

Data Ingestion (using lake flow connect):

Earlier the organization have to use a lot of third-party tools and inhouse solutions for the data ingestions.

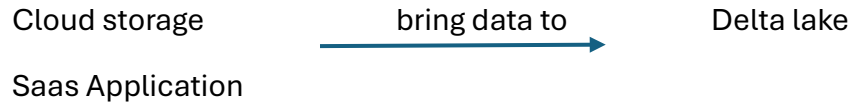
The data might be residing in

cloud storages → adls gen2, amazon s3, databricks volumes

Databases

Saas Applications

With lakeflow connect we have efficient ingestion pipelines **all within** databricks.



You ingest to delta lake , And once data is in delta lake you can do ETL.

The most common ingestion pattern is Medallion architecture.



Lakeflow connect:

Different types of connectors:

- Upload files from local storage to volume.
- Standard connectors (ingest from cloud storage into your delta lake)
- Managed connectors (ingest from SaaS applications / Databases)

Standard connectors:

Different kind of ingestion modes:

Batch ingestion: (All data is re-ingested each time)

Incremental batch: (all new data is ingested , previously loaded records are skipped automatically.)

Incremental streaming – continuously load data rows or batches of data rows as it is generated. So that you can query as it arrives in near real time.

In next session we will see how to do data ingestion from cloud storage.

Cloud Storage Ingestion: CTAS

Data Ingestion from cloud storage. We have options of

- Create Table as select
- Copy Into
- Auto Loader

Lakeflow connect standard connectors are CTAS,copyinto,Autoloader.

Convert raw file into delta tables.

1 CTAS (CREATE TABLE AS)

- Best for one-time loads or quick ad-hoc analysis
- Not idempotent — re-running can duplicate data
- Schema evolution is manual or inferred
- Higher latency but super simple to use

2 COPY INTO

- Ideal for incremental batch loads and repeatable jobs
- Idempotent — safe to re-run without duplicates
- Supports schema evolution with manual config
- Moderate latency, straightforward SQL syntax

3 Auto Loader

- Perfect for continuous streaming and massive-scale ingestion
- Idempotent with built-in checkpointing
- Fully automated schema evolution
- Low latency (near real-time) but requires slightly more setup ---

💡 Pro Tip: - Small, quick loads → CTAS - Repeatable scheduled loads → COPY INTO - Continuous or large-scale ingestion → Auto Loader for automation + scale

CTAS

create table as select *

All the data is re ingested each time.

```
%sql
use catalog deltalake_demo;
```

```
%sql
create volume deltalake_demo.default.raw;
```

%fs

mkdirs /Volumes/deltalake_demo/default/raw/orderhistory

```
# -- in below command we can able to view what are the files are present under the
path
# %sql
# LIST '/Volumes/deltalake_demo/default/raw/orderhistory'

# %fs
# ls '/Volumes/deltalake_demo/default/raw/orderhistory'
```

Upload order1.csv files into volume.

```
%sql
SELECT * from CSV.`/Volumes/deltalake_demo/default/raw/orderhistory`
```

```
%sql
SELECT * from
read_files
(
    '/Volumes/deltalake_demo/default/raw/orderhistory',
```

```
format=>'csv'  
);
```

Using CTAS

```
%sql  
drop table if exists orders;  
create table orders  
as  
SELECT * from  
read_files  
(  
    '/Volumes/deltalake_demo/default/raw/orderhistory',  
    format=>'csv'  
);
```

The above table creation is inferring schema automatically but I need to give fixed column datatypes.

orders3.csv file has extra column. order_amount.

order4.csv file has some malformed data. when I run read_files command it will ingest all the files from beginning.

We need to give our Schema fixed datatypes

```
%sql  
drop table if exists orders;  
create table orders  
as  
SELECT * from  
read_files  
(  
    '/Volumes/deltalake_demo/default/raw/orderhistory',  
    format=>'csv',
```

```

schema => 'order_id bigint,order_date string,customer_id bigint,order_status
string,_rescued_data string',
header => true,
mode => 'PERMISSIVE',
columnNameofCorruptRecord => '_rescued_data'
);

```

07:39 AM (2s) 15 SQL

```
%sql
SELECT * from orders;

-- take a screen shot of below output
```

See performance (1) ✓ Checked for improvements

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [order_id: long, order_date: string ... 3 more fields]

Table +

	order_id	order_date	customer_id	order_status	_rescued_data
13	2003	2026-08-14 01:01:00	5012	Pending	null
14	2004	2026-08-08 16:38:00	5004	Pending	null
15	2005	2026-08-14 07:28:00	5076	Shipped	null
16	1001	2026-08-17 10:00:00	null	Completed	1001,2026-08-17 10:00:00,CUST892,Completed
17	1002	null	null	Pending	1002,,CUST411,Pending
18	1003	InvalidDate	null	Shipped	1003,InvalidDate,CUST512,Shipped
19	1004	2026-08-17 11:15:00	null	NotANumber	1004,2026-08-17 11:15:00,CUST301,NotANumber
20	1005	2026-08-17 11:30:00	null	Cancelled	1005,2026-08-17 11:30:00,CUST105,Cancelled,ExtraValue
21	1006	2026-08-17 12:00:00	null	null	1006,2026-08-17 12:00:00
22	3001	2026-01-21 03:01:00	null	Shipped	3001,2026-01-21 03:01:00,CUST859,Shipped,130.0
23	3002	2026-01-05 23:06:00	null	Completed	3002,2026-01-05 23:06:00,CUST792,Completed,299.34
24	3003	2026-01-02 00:05:00	null	Pending	3003,2026-01-02 00:05:00,CUST323,Pending,257.62
25	3004	2026-01-01 17:12:00	null	Cancelled	3004,2026-01-01 17:12:00,CUST833,Cancelled,118.02
26	3005	2026-01-19 08:51:00	null	Completed	3005,2026-01-19 08:51:00,CUST990,Completed,381.82

Observe _rescued_data column.

Mode -> PERMISSIVE :: the mode defines how spark should behave when it encounters bad or malformed records.

PERMISSIVE → keeps the job running, and put the bad rows into _rescued_data column.

Dropmalformed → silently drops bad records

Failfast → aborts on the first bad row.

When *Not* to Use CTAS

While CTAS is perfect for batch processing, it has specific functional trade-offs that you will need to watch out for in real-world pipelines:

- **No Incremental Loading:** Every time you run a CTAS statement, it creates or overwrites the entire table. It cannot seamlessly identify and append *only new files* added to the cloud bucket. **Each time it trigger full load.**
- **No Schema Evolution:** If your source file structure changes tomorrow (e.g., a new column is added), a basic CTAS command will either fail or require you to completely recreate the table.

COPY INTO – DEMO 1:

- 🚦 Loads data from cloud file location into delta lake.
- 🚦 This is retryable and idempotent operation. (Exactly once processing)
- 🚦 **idempotency** means that running a data pipeline, query, or job multiple times with the same input data produces the exact same final result without causing duplicates, data corruption, or unintended side effects.
- 🚦 **Incremental batch:** *files In the source location that have already been loaded are skipped.*

■ *Take notes from databricks notebook*

Autoloader:

Lakeflow connect standard connectors

- CTAS
- copy into (incremental batch)

Scenarions which copy command cannot handle:

- what if a new file is coming in every 5 minutes, every 2 min or even more frequently.
- if millions of new files are coming each hour, or you have to backfill(historical data) billions of files.

thats where autoloader is needed.its a more advance technique then copy into to incrementally and efficiently process new data files as they arrive in cloud storage.

- autoloader can handle both incremental batch and streaming.
- you can use autoloader to process billions of files to migrate or backfill a table.
autoloader scales to support near real time ingestion of millions of files per hour.
- autoloader can load data files from any of the cloud source.
ex: amazon s3 ,adls gen2, GCS

```
%sql  
use catalog deltalake_demo;
```

```
landing_zone="/Volumes/deltalake_demo/default/raw/"  
orders_data = landing_zone + "/orderhistory"  
checkpoint_path =landing_zone +"/orders_checkpoint"
```

The `.option("cloudFiles.inferColumnTypes", "true")` configuration is used in **Databricks Auto Loader** (`format("cloudFiles")`) to force Apache Spark to **automatically discover and apply the correct data types** for your incoming data fields.

By default, when Auto Loader reads text-based file formats like JSON, CSV, or XML, it treats **every column as a string (StringType)**. It does this to prevent runtime data type mismatch crashes.

```
orders_df=(spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format","csv")
    .option("cloudFiles.inferSchema","true")
    .option("cloudFiles.inferColumnTypes","true")
    .option("cloudFiles.schemaLocation",checkpoint_path)
    .load(orders_data))
```

```
%sql
drop table if exists deltalake_demo.default.ordersdelta;
```

Write stream is a unified API for both batch and streaming

for streaming we can use

```
.trigger(processingTime = 10 seconds) (this is streaming type)

.trigger(availableNow=True) (this is batch style when we run the job trigger)
```

Write Stream is a unified API for both and streaming...

```
orders_df.writeStream \
    .format("delta") \
    .option("checkpointLocation",checkpoint_path) \
    .option("mergeSchema","true") \
    .outputMode("append") \
    .trigger(availableNow=True) \
    .toTable("deltalake_demo.default.ordersdelta")
```

```
%sql
Select * from deltalake_demo.default.ordersdelta;

-- we have resuced_column.
```

order3.csv -- with one extra column. Order_amount column.

```
orders_df.writeStream \
    .format("delta") \
    .option("checkpointLocation",checkpoint_path) \
    .option("mergeSchema","true") \
    .outputMode("append") \
    .trigger(availableNow=True) \
    .toTable("deltalake_demo.default.ordersdelta")
```

-Encountered unknown fields during parsing: [order_amount], which can be fixed by an automatic retry: true

to resolve above error first we need to run the readstream part then writestream...

Yesterday (2s) 15

```
%sql
select * from deltalake_demo.default.ordersdelta;
```

> [See performance \(1\)](#) ✓ Checked for improvements

	r_status	rescued_data	1.2 order_amount
1		{"order_date":"1/21/2026 3:01","_file_path":"/Volumes/deltalake_demo/default/raw/orderhistory/orders3.csv"}	130
2	ed	{"order_date":"1/5/2026 23:06","_file_path":"/Volumes/deltalake_demo/default/raw/orderhistory/orders3.csv"}	299.34
3		{"order_date":"1/2/2026 0:05","_file_path":"/Volumes/deltalake_demo/default/raw/orderhistory/orders3.csv"}	257.62
4	d	{"order_date":"1/1/2026 17:12","_file_path":"/Volumes/deltalake_demo/default/raw/orderhistory/orders3.csv"}	118.02
5	ed	{"order_date":"1/19/2026 8:51","_file_path":"/Volumes/deltalake_demo/default/raw/orderhistory/orders3.csv"}	381.82
6	ed	null	null
7	d	null	null
8	ed	null	null
9	ed	null	null
10	ed	null	null
11		null	null
12		null	null
13	ed	null	null
14		null	null
15			

20 rows 1.857s runtime Refreshed Yesterday

This result is stored as `_sqldf` and can be used in other [Python](#) and [SQL](#) cells. [Send feedback](#)

lets upload order4.csv --> datatype mismatch

```
orders_df.writeStream \  
  .format("delta") \  
  .option("checkpointLocation",checkpoint_path) \  
  .option("mergeSchema","true") \  
  .outputMode("append") \  
  .trigger(availableNow=True) \  
  .toTable("deltalake_demo.default.ordersdelta")  
  
# the job is didn't fail
```

```
%sql  
select * from deltalake_demo.default.ordersdelta;  
  
-- when the data miss match happens the job won't fail the data comes under  
_rescued_data column like below  
{ "order_date": "InvalidDate", "customer_id": "CUST512", "_file_path": "/Volumes/deltalake_  
demo/default/raw/orderhistory/order4.csv" }
```

what happens if a new column comes ?

When a new file like order3.csv arrives with an extra column, Databricks Auto Loader handles it automatically through a process called **Schema Evolution**.

Here is exactly how Auto Loader reacts and adapts to new columns:

The Auto Loader Workflow for New Columns

1. **Automatic Detection:** Auto Loader detects the new column in order3.csv as soon as it processes the file.
2. **Stream Interruption (By Design):** The stream will intentionally stop and exit with a `UnknownFieldException` error. This is normal and expected behavior. [[1](#), [2](#)]
3. **Schema Schema Updated:** Behind the scenes, Auto Loader updates the schema file stored in your configured `.option("cloudFiles.schemaLocation", "...")` path to include the new column.

4. **Stream Resumed:** When your stream restarts (manually or via a workflow retry policy), it reads the updated schema. It will now successfully process order3.csv and include the new column without losing any data

what happen with there is schema mismatch ?

When a **data type schema mismatch** occurs—for example, your schema expects age to be an Integer, but a new file passes a text string like "Twenty"—Databricks Auto Loader prevents data loss by routing that specific mismatched cell into a hidden column called **_rescued_data**.

Unlike a new column addition, a standard data type mismatch **will not crash or stop your stream** by default.

How Auto Loader Rescues Mismatched Data

Instead of dropping the invalid row or failing the cluster, Auto Loader completes the record ingestion but isolates the offending data:

1. **Table Placement:** The main column (age) is populated with a NULL value for that specific row.
2. **The Rescue Blob:** The original mismatched value is written to the **_rescued_data** column as a JSON string.
3. **Audit Trail:** This JSON blob also captures the absolute file path, allowing you to trace exactly which file sent the malformed data.

Visual Example of a Mismatch

If order4.csv sends a bad string data type into an integer column:

order_id	price (Integer)	_rescued_data (String / JSON Blob)
1001	45	<i>null</i>
1002	<i>null (Mismatched value rescued)</i>	{"price": "FREE_PROMO", "_file_path": "dbfs:/orders/order4.csv"}

How to Query the rescued Data:

```
SELECT  
  
  order_id,  
  
  _rescued_data.price AS original_raw_price,  
  
  _rescued_data._file_path AS source_file  
  
FROM my_bronze_table  
  
WHERE _rescued_data IS NOT NULL
```

how does autoloader track the ingestion progress ?

as files are discovered, their metadata is persisted in a scalable key value store (RocksDB)

the RocksDB is created in the checkpoint location

/Volumes/deltalake_demo/default/raw/orders_checkpoint/sources/0/rocksdb/

this key value store ensures that data is processed exactly once.

in case of failures , autoloader can resume from where it left off by information stored in the checkpoint location.

this is what helps us to get exacty once semantics.

For the streaming part severless won't support .. we need to create a classic compute

```
orders_df=(spark.readStream  
  .format("cloudFiles")  
  .option("cloudFiles.format","csv")  
  .option("cloudFiles.inferSchema","true")  
  .option("cloudFiles.inferColumnTypes","true")  
  .option("cloudFiles.schemaLocation",checkpoint_path)  
  .load(orders_data))
```

```
# the major difference between batch and streaming is the trigger option....
orders_df.writeStream \
    .format("delta") \
    .option("checkpointLocation",checkpoint_path) \
    .option("mergeSchema","true") \
    .outputMode("append") \
    .trigger(processingTime="10 seconds") \
    .toTable("deltalake_demo.default.ordersdelta")
```

New Session: working with Schemas

- ✚ how schema inference works internally.
- ✚ what if you want to manually define the schema
- ✚ Schema hints
- ✚ add new metadata columns

1.how schema inference works internally.

basically when every you infer the schema. it will check

1000 files or 50GB of data whatever comes first.

1000 files each of 1 MB = 1 GB

10 files of 5 gb each = 50 GB

what ever comes first that will be infered.

properties are:

cloudFiles.schemaInference.sampleSize.numBytes;

cloudFiles.schemaInference.sampleSize.numFiles;

2 . what if you want to manually define the schema

```
from pyspark.sql.types import
StructType, StructField, StringType, DoubleType, TimestampType, IntegerType

orders_schema=StructType([
    StructField("order_id", IntegerType(), True),
    StructField("order_date", TimestampType(), True),
    StructField("customer_id", IntegerType(), True),
    StructField("order_status", StringType(), True),
])
```

when we define the schema manually the rescued data column is not there.. we need to manually add the rescued column.

```
orders_df=(spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("rescuedDataColumn", "_rescued_data")
    .schema(orders_schema)
    .option("cloudFiles.schemaLocation", checkpoint_path)
    .load(orders_data))
```

```
orders_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", checkpoint_path) \
    .option("mergeSchema", "true") \
    .outputMode("append") \
    .trigger(availableNow=True) \
    .toTable("deltalake_demo.default.ordersdelta")
```

SQL 32 02:46 PM (6s) [See performance \(1\)](#) [Checked for improvements](#)

	order_id	order_date	customer_id	order_status	_rescued_data
1	1000	2026-08-08T14:37:00.000+00:...	541	Completed	null
2	1001	2026-08-09T00:48:00.000+00:...	508	Cancelled	null
3	1002	2026-08-06T22:27:00.000+00:...	502	Completed	null
4	1003	2026-08-11T08:09:00.000+00:...	548	Completed	null
5	1004	2026-08-07T10:06:00.000+00:...	518	Completed	null
6	1005	2026-08-03T12:06:00.000+00:...	516	Pending	null
7	1006	2026-08-12T11:38:00.000+00:...	515	Pending	null
8	1007	2026-08-09T01:46:00.000+00:...	509	Completed	null
9	1008	2026-08-15T17:07:00.000+00:...	548	Pending	null
10	1009	2026-08-13T02:35:00.000+00:...	507	Cancelled	null

10 rows 5.767s runtime Refreshed 3h ago

lets upload order3.csv file -- with one extra column and run writestream

what if system infer the schema and what if I give my own schema in autoloader databaricks.. when the new file arrives with one extra column

Scenario 1: Auto Loader Infers the Schema

If you use schema inference (cloudFiles.inferSchema), Auto Loader automatically adapts to the new column using a feature called Schema Evolution.

The New Column: Auto Loader detects the extra column in the new file.

Stream Behavior: The streaming job will intentionally fail and stop.

The Fix: It logs the new column to a schema directory. When the stream restarts (manually or via a retry policy), it reads the updated schema.

The Result: The new column is seamlessly added to your target table. Existing rows will show null for this new column.

Scenario 2: You Provide Your Own Schema

If you explicitly define the schema in your code, Auto Loader strictly enforces that structure.

The New Column: Auto Loader ignores the extra column during normal processing because it is not part of your defined schema.

The "Rescued Data" Column: Instead of dropping the data, Auto Loader places the extra column and its value into a special hidden column called `_rescued_data` (stored as a JSON string).

Stream Behavior: The streaming job keeps running without any failures or interruptions.

The Result: Your target table schema remains unchanged. You do not lose the new data; it is safely parked in the `_rescued_data` column so you can audit or recover it later.

<u>Feature</u>	<u>Scenario 1: Inferred Schema</u>	<u>Scenario 2: Provided Schema</u>
Stream Action	Fails once, updates schema, restarts	Keeps running smoothly
Target Table	Automatically gets the new column	Target schema stays the same
Data Location	Moves to the new column	Moves to <code>_rescued_data</code>

lets upload order4.csv file -- the datatype mismatch and run writestream

```
orders_df.writeStream \  
  .format("delta") \  
  .option("checkpointLocation",checkpoint_path) \  
  .option("mergeSchema","true") \  
  .outputMode("append") \  
  .trigger(availableNow=True) \  
  .toTable("deltalake_demo.default.ordersdelta")
```

the above writestream it behaves same like order3.csv

3.Schema hints

Schema hints in Databricks are an overriding mechanism used to force specific data types on certain columns during automatic schema inference without needing to define the entire dataset's schema manually..

They are primarily used in Databricks Auto Loader (cloudFiles) and the from_json() pipeline function to fix sampling errors while preserving dynamic schema evolution.

```
%sql
drop table deltalake_demo.default.ordersdelta;
```

```
from pyspark.sql.functions import col,current_timestamp
orders_df=(spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format","csv")
    .option("cloudFiles.inferSchema","true")
    .option("cloudFiles.inferColumnTypes","true")
    .option("cloudFiles.schemaHints","order_id BIGINT,order_date string")
    .option("cloudFiles.schemaLocation",checkpoint_path)
    .load(orders_data)
    .withColumn("file_name",col("_metadata.file_path"))
    .withColumn("ingestion_time",current_timestamp())
)
```

```
order_id:long
order_date:string
customer_id:integer
order_status:string
order_amount:double
_rescued_data:string
file_name:string
ingestion_time:timestamp
```

```
orders_df.writeStream \
    .format("delta") \
    .option("checkpointLocation",checkpoint_path) \
    .option("mergeSchema","true") \
    .outputMode("append") \
    .trigger(availableNow=True) \
    .toTable("deltalake_demo.default.ordersdelta")
```

03:26 PM (2s) 45

```
%sql
SELECT * from deltalake_demo.default.ordersdelta;
```

> [See performance \(1\)](#) ✓ Checked for improvements

	order_status	rescued_data	file_name	ingestion_time
1	Completed	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
2	Cancelled	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
3	Completed	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
4	Completed	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
5	Completed	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
6	Pending	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
7	Pending	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
8	Completed	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
9	Pending	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
10	Cancelled	null	/Volumes/deltalake_demo/default/raw/orderhistory/order1.c...	2026-08-20T09:55:12.529+00:...
11	Pending	null	/Volumes/deltalake_demo/default/raw/orderhistory/order2.c...	2026-08-20T09:56:27.745+00:...
12	Completed	null	/Volumes/deltalake_demo/default/raw/orderhistory/order2.c...	2026-08-20T09:56:27.745+00:...
13	Pending	null	/Volumes/deltalake_demo/default/raw/orderhistory/order2.c...	2026-08-20T09:56:27.745+00:...
14	Pending	null	/Volumes/deltalake_demo/default/raw/orderhistory/order2.c...	2026-08-20T09:56:27.745+00:...
15				

first run the readstream then writestream

4. last session: Schema Evolution Modes

In Databricks Auto Loader, `cloudFiles.schemaEvolutionMode` dictates how your data ingestion pipeline handles changes, additions, or mismatches in the incoming data schema....

Schema evolution Modes

- none
- fail on new columns
- rescue
- add new columns (default)

none:

====

Does not evolve the schema, new columns are ignored, and data is not rescued unless the `rescuedDataColumn` option is set. Stream does not fail due to schema changes.

```
%sql
```

```
drop table if exists deltalake_demo.default.ordersdelta;
```

```
orders_df=(spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format","csv")
    .option("cloudFiles.inferSchema","true")
    .option("cloudFiles.inferColumnTypes","true")
    .option("cloudFiles.schemaLocation",checkpoint_path)
    .option("cloudFiles.schemaEvolutionMode","none")
    .load(orders_data))
```

```
orders_df.writeStream \
    .format("delta") \
    .option("checkpointLocation",checkpoint_path) \
    .option("mergeSchema","true") \
    .outputMode("append") \
    .trigger(availableNow=True) \
    .toTable("deltalake_demo.default.ordersdelta")
```

"Ignore the Noise" (Fixed-Purpose Consumer)

- **Goal:** You only care about a specific subset of columns. You do not want your storage or compute resources spent on handling extra data attributes.
- **Behavior:** The pipeline runs continuously and quietly drops any new fields or mismatched data it encounters.
- **Auto Loader Configuration:**

.option("cloudFiles.schemaEvolutionMode", "none")

- **Result:** Any data field not explicitly defined in your initial schema layout is permanently ignored during the ingestion process.

- lets upload order3.csv -- with one extra column
- the job won't fail because we mention mode as ****_None_****

```
orders_df.writeStream \  
  .format("delta") \  
  .option("checkpointLocation",checkpoint_path) \  
  .option("mergeSchema","true") \  
  .outputMode("append") \  
  .trigger(availableNow=True) \  
  .toTable("deltalake_demo.default.ordersdelta")
```

[fail on new columns](#)

stream fails, stream does not restart unless the provided schema is updated or the offending data file is removed.

"Lock it Down" (Financial & Regulated Data)

- **Goal:** Strict schema enforcement. Unannounced changes mean something upstream is broken, and you need to investigate immediately.
- **Behavior:** The pipeline throws a hard error and stops processing immediately when any row violates the schema layout.

Auto Loader Configuration:

.option("cloudFiles.schemaEvolutionMode", "failOnNewColumns")

- **Result:** Your operations team receives an instant alert. Bad or altered data is completely blocked from entering your clean database layers until a data engineer updates the official schema definition.

[Rescue:](#)

schema is never evolved and stream does not fail due to schema changes.

all new columns are recorded in the rescued data column.

Ex:

"Never Stop Running" (Production-Safe):

Goal: You want absolute uptime. A bad row or schema drift must never crash your production jobs.

Behavior: The pipeline keeps running smoothly. It processes valid rows as usual, while capturing the corrupted or unexpected fields into a special column for later analysis.

`.option("cloudFiles.schemaEvolutionMode", "rescue")`

Result: Databricks automatically creates a `_rescued_data` column. If a column expects an integer but receives a text string like "abc", or if an entirely new field appears, it is cleanly saved in this column as a JSON string. No data is lost, and the stream never stops..

addNewColumns:

Automatic_retry: true

stream fails, new columns are added to the schema.. existing columns do not evolve data types..

"Adapt and Grow" (Data Science & Analytics)

- **Goal:** You want to capture new data attributes automatically as upstream business systems evolve.
- **Behavior:** When a new column appears, the stream temporarily pauses, updates its schema metadata, and immediately resumes.
- `.option("cloudFiles.schemaEvolutionMode", "addNewColumns")`

Result: The system dynamically appends the new fields to your target schema. *Note: If a row is flat-out malformed (e.g., corrupted JSON structural syntax), it will still be dropped or sent to the `_rescued_data` column depending on your format settings.*

Mode	Pipeline Behavior on New Columns	Schema Updates	Primary Use Case
addNewColumns (Default)	Stream fails temporarily; must be restarted.	Automatically appends new columns to the schema tracking file.	Flexible environments where adding new tracking columns is normal and safe.
rescue	Stream keeps running safely without failing.	Schema is never changed. Unknown fields are routed into a <code>_rescued_data</code> column.	Production environments requiring maximum uptime, audit trails, or regulatory oversight.
failOnNewColumns	Stream fails permanently until manual action is taken.	Schema is not updated in metadata.	Regulated pipelines where upstream drift signals bad data or unexpected breaking bugs.
none	Stream keeps running without failing.	Schema remains fixed. New columns are completely dropped and ignored.	Legacy consumption where only a fixed subset of attributes matters and structural updates are irrelevant.